U N I W E R S Y T E T Ś L Ą S K I
W K A T O W I C A C H

Informatyka stacjonarna I stopnia

Maksym Litosh

Informatyczny asystent zarządzania certyfikatami i subskrypcjami (CertiSub Assistant)

Praca inżynierska

napisana pod kierunkiem Jarosław Utracki

Katowice 2026

Spis treści

1. Wstęp 5
1.1. Analiza problemu zarządzania certyfikatami i subskrypcjami 5
1.2. Charakterystyka problemu w środowisku firmowym i prywatnym 5
1.3. Proces odnowień i cykl życia subskrypcji IT 5
1.4. Rola technologii informatycznych w monitorowaniu płatności cyklicznych 7
2. Badanie rynkowe i przegląd istniejących rozwiązań 8
2.1. Problem rozproszonego zarządzania subskrypcjami 8
2.2. Analiza istniejących narzędzi i platform 8
2.3. Technologie informatyczne stosowane w systemach zarządzania subskrypcjami 9
2.3.1. Architektura klient–serwer 9
2.3.2. Relacyjne bazy danych 9
2.3.3. Technologie frontendowe 11
2.3.4. Uwierzytelnianie hasłem i weryfikacja adresu e-mail 11
2.4. Systemy alertów i powiadomień o terminach płatności 11
2.5. Studium wykonalności: PHP/LAMP a Java/Spring Boot 12
2.6. Wnioski z analizy i uzasadnienie realizacji projektu 12
3. Założenia projektowe aplikacji 14
3.1. Cel i zakres projektu 14
3.2. Wymagania funkcjonalne 14
3.3. Wymagania niefunkcjonalne 15
3.4. Role użytkowników i przypadki użycia 16
3.5. Architektura ogólna systemu 17
4. Projekt bazy danych 19
4.1. Model pojęciowy danych 19
4.2. Diagram ERD 19
4.3. Struktura bazy danych i relacje między encjami 21
4.4. Integralność i bezpieczeństwo danych 23
5. Projekt i implementacja aplikacji webowej 25
5.1. Wykorzystane technologie i narzędzia 25
5.2. Implementacja warstwy backendowej (PHP) 25
5.3. Implementacja warstwy frontendowej (Vue.js, Tailwind CSS) 26
5.4. Implementacja dashboardu i formularzy 27
5.5. System logowania hasłem i weryfikacji adresu e-mail 35
6. Moduł zadań, zaproszeń i przypomnień e-mail 38
6.1. Założenia procesu odnowień i zadań ToDo 38
6.2. Szablony wiadomości, załączniki i renderowanie treści 39
6.3. Mechanizm cron, wysyłka zaproszeń i przypomnień 40
6.4. Integracja modułu z aplikacją webową i rejestr zdarzeń 42
6.5. Wnioski o certyfikat przesłane e-mailem 43
6.6. Dane firm z otwartych rejestrów publicznych 44
6.7. Powiadomienia wewnętrzne między kontami 45
7. Testowanie i ocena systemu 47
7.1. Scenariusze testowe 47
7.2. Testy funkcjonalne 47
7.3. Testy integracyjne i bezpieczeństwa logowania 49

7.4. Ocena użyteczności interfejsu dashboardu ... 50
7.5. Analiza uzyskanych wyników ... 51
8. Dokumentacja użytkownika ... 53
8.1. Wymagania i uruchomienie ... 53
8.2. Logowanie, konta i role ... 53
8.3. Praca z ewidencją ... 53
8.4. Proces odnowienia krok po kroku ... 54
8.5. Raporty, wymiana danych i wnioski e-mail ... 54
8.6. Administracja ... 54
8.7. Rozwiązywanie problemów ... 55
9. Zakończenie ... 56
9.1. Podsumowanie pracy ... 56
9.2. Kierunki dalszego rozwoju ... 57
Bibliografia ... 58
Spis rysunków ... 60
Spis tabel ... 61
Załączniki ... 62
Załącznik A. Oficjalny opis pracy ... 62
Załącznik B. Macierz uprawnień ról ... 64
Załącznik C. Struktura katalogów repozytorium ... 64

1. Wstęp

1.1. Analiza problemu zarządzania certyfikatami i subskrypcjami

W ostatnich latach obserwuje się gwałtowny wzrost liczby usług cyfrowych opartych na modelu subskrypcyjnym oraz liczby certyfikatów elektronicznych towarzyszących codziennej działalności organizacji. Przedsiębiorstwa korzystają z szerokiego spektrum rozwiązań podlegających okresowej odnowie: kwalifikowanych podpisów i pieczęci elektronicznych, certyfikatów SSL/TLS, certyfikatów code signing, rejestracji domen, licencji SaaS oraz planów wsparcia chmurowego. Każda z tych pozycji wiąże się z konkretną datą wygaśnięcia, numerem seryjnym nadanym przez wystawcę, kosztem oraz odpowiedzialnością po stronie wyznaczonego użytkownika certyfikatu (beneficjenta) i podmiotu finansującego (płatnika).

Proces monitorowania terminów bywa skomplikowany i czasochłonny. Dane o certyfikatach i subskrypcjach często rozproszone są pomiędzy arkuszami kalkulacyjnymi, skrzynkami pocztowymi, portalami dostawców oraz dokumentacją papierową. Powoduje to utratę ciągłości usług, ryzyko przeoczenia terminu odnowienia oraz konieczność wielokrotnych konsultacji między działem IT, działem prawnym a finansami.

1.2. Charakterystyka problemu w środowisku firmowym i prywatnym

W środowisku firmowym zarządzanie certyfikatami i subskrypcjami wymaga uwzględnienia wielu czynników: typu certyfikatu, daty wygaśnięcia, wymaganego czasu na odnowienie, użytkownika certyfikatu, podmiotu płacącego, statusu płatności oraz priorytetu odnowienia. Nieodnowiony certyfikat SSL może spowodować awarie API i utratę zaufania klientów, wygaśnięty certyfikat kwalifikowany — wstrzymanie podpisywania dokumentów urzędowych, a opóźniona płatność za SaaS — zawieszenie licencji i spadek produktywności zespołu.

Pierwotna koncepcja projektu zakładała objęcie jednym systemem także segmentu prywatnego, w którym użytkownicy indywidualni mierzą się ze zjawiskiem „zmęczenia subskrypcjami” (subscription fatigue) — wieloma usługami miesięcznymi (streaming, muzyka, chmura, narzędzia AI) bez jednego widoku łącznych wydatków. W toku prac analitycznych stwierdzono jednak, że model danych, krąg odbiorców oraz wymagania bezpieczeństwa rejestru firmowego (beneficjenci, płatnicy, role personelu, RODO) i osobistego menedżera subskrypcji są na tyle odmienne, że ich połączenie w jednej bazie i jednym panelu szkodziłoby przejrzystości obu rozwiązań. Z tego powodu moduł prywatny został wydzielony do osobnej aplikacji („Menedżer Subskrypcji”, decyzja projektowa D1, szerzej w podrozdziale 3.1), a niniejsza praca opisuje docelowo firmowy rejestr certyfikatów i subskrypcji.

1.3. Proces odnowień i cykl życia subskrypcji IT

Cykl życia certyfikatu lub subskrypcji obejmuje rejestrację pozycji, aktywne użytkowanie, monitorowanie terminu odnowienia, wysyłkę zaproszenia do beneficjenta lub płatnika, realizację odnowienia (wystawienie nowego certyfikatu i zarchiwizowanie poprzedniego)

albo świadome porzucenie procesu. W praktyce organizacje stosują różne cykle rozliczeniowe: miesięczne (licencje SaaS), roczne (certyfikaty SSL, domeny) oraz wieloletnie (certyfikaty kwalifikowane, umowy enterprise).

Współczesne podejście oparte wyłącznie na arkuszach kalkulacyjnych nie zapewnia alertów, wersjonowania zmian ani wieloużytkownikowego dostępu. Konieczne staje się stworzenie narzędzia informatycznego wspierającego proces podejmowania decyzji i umożliwiającego szybką identyfikację pozycji wymagających natychmiastowego działania.

1.4. Rola technologii informatycznych w monitorowaniu płatności cyklicznych

Dynamiczny rozwój aplikacji webowych, relacyjnych baz danych oraz systemów powiadomień e-mail stwarza nowe możliwości centralizacji informacji o subskrypcjach. Wykorzystanie architektury klient–serwer, prepared statements, uwierzytelniania opartego na haśle z weryfikacją adresu e-mail oraz reaktywnych interfejsów (Vue.js) pozwala na integrację rozproszonych danych i usprawnienie procesu monitorowania terminów.

Celem niniejszej pracy inżynierskiej jest zaprojektowanie i implementacja aplikacji webowej CertiSub Assistant wspierającej ewidencję certyfikatów i subskrypcji organizacji wraz z ich użytkownikami (beneficjentami) i płatnikami, z wykorzystaniem nowoczesnych technologii informatycznych. Efektem końcowym pracy jest działający prototyp systemu umożliwiający prowadzenie rejestru certyfikatów, automatyczne wykrywanie zbliżających się terminów odnowienia, prezentację alertów i statystyk w dashboardzie, wysyłkę zaproszeń i przypomnień e-mail na podstawie szablonów oraz raportowanie i wymianę danych (import/eksport CSV, XML i wiadomości e-mail).

2. Badanie rynkowe i przegląd istniejących rozwiązań

2.1. Problem rozproszonego zarządzania subskrypcjami

Zarządzanie certyfikatami i subskrypcjami IT stanowi istotny problem organizacyjny zarówno w małych i średnich przedsiębiorstwach, jak i w dużych firmach z rozbudowaną infrastrukturą. Każdego roku organizacje odnawiają dziesiątki certyfikatów kwalifikowanych, certyfikatów SSL, licencji SaaS oraz umów wsparcia, a odpowiedzialność za poszczególne pozycje bywa rozproszona pomiędzy dział IT, dział prawny i finanse. Jednocześnie informacje o terminach, kosztach i osobach odpowiedzialnych pozostają rozproszone i trudne do zestawienia w jednym miejscu.

Proces monitorowania odnowień jest często rozproszony i opiera się na korzystaniu z wielu różnych źródeł: arkuszy Excel, skrzynek e-mail, portali dostawców (AWS, Microsoft 365, urzędów certyfikacji) oraz papierowej dokumentacji. Powoduje to trudności w szybkiej identyfikacji pozycji zbliżających się do wygaśnięcia oraz utrudnia przypisanie odpowiedzialności za konkretne odnowienie.

2.2. Analiza istniejących narzędzi i platform

W celu lepszego zrozumienia problematyki zarządzania certyfikatami i subskrypcjami przeprowadzono analizę wybranych rozwiązań dostępnych obecnie na rynku. Do badania wybrano kategorie narzędzi umożliwiających śledzenie wydatków subskrypcyjnych, zarządzanie cyklem życia certyfikatów oraz zarządzanie zasobami IT, uwzględniając zarówno platformy międzynarodowe, jak i usługi polskich centrów certyfikacji obsługujących podpisy i pieczęcie kwalifikowane.

Analizie poddano następujące rozwiązania:

- arkusze kalkulacyjne (Excel / Google Sheets) — powszechne w MŚP,
- narzędzia zarządzania cyklem życia certyfikatów (Keyfactor Command, DigiCert Trust Lifecycle Manager, Sectigo Certificate Manager) — segment enterprise,
- systemy ITSM/CMDB (ServiceNow, Lansweeper) — ewidencja zasobów IT bez specjalizacji w certyfikatach,
- panele polskich centrów certyfikacji kwalifikowanej (Certum, KIR — Szafir, EuroCert) — ograniczone do własnych certyfikatów danego wystawcy,
- portale dostawców subskrypcji (AWS, Microsoft 365, operatorzy domen) — dane w silosach,
- narzędzia monitorowania certyfikatów SSL (SSL Labs, certbot, panele hostingowe).

Wybrane rozwiązania zostały porównane pod względem funkcjonalności istotnych z punktu widzenia projektowanego systemu CertiSub Assistant.

Funkcjonalność	Excel	Cert. LCM	ITSM	Centra cert.	CertiSub
Centralny rejestr certyfikatów	Tak	Tak	Tak	Nie	Tak
Ewidencja beneficjentów i płatników	Ręcznie	Nie	Częściowo	Nie	Tak
Alerty terminów odnowień	Nie	Tak	Tak	Ograniczone	Tak
Zadania ToDo z priorytetami	Nie	Tak	Tak	Nie	Tak
Zaproszenia e-mail z załącznikami	Nie	Ograniczone	Tak	Nie	Tak
Dashboard KPI i priorytety	Nie	Tak	Tak	Nie	Tak
Wyszukiwarka z powiązaniami	Ograniczone	Tak	Tak	Nie	Tak
Logowanie hasłem + weryfikacja e-mail	Nie	Tak	Rzadko	Nie	Tak
Import / eksport CSV, XML, EML	Tak*	Ograniczone	Tak	Nie	Tak
Niski koszt wdrożenia	Tak	Nie	Nie	Tak	Tak

Tabela przedstawia porównanie najważniejszych funkcji analizowanych rozwiązań (Cert. LCM — narzędzia klasy Certificate Lifecycle Management, np. Keyfactor Command, DigiCert Trust Lifecycle Manager; Centra cert. — panele polskich centrów certyfikacji kwalifikowanej, np. Certum, KIR Szafir, EuroCert; * Excel eksportuje/importuje wyłącznie CSV, bez XML ani poczty EML).

Na podstawie przeprowadzonej analizy można zauważyć, że dostępne rozwiązania albo są zbyt ogólne i wymagają ręcznego utrzymania (arkusze), albo zbyt drogie i złożone jak na potrzeby małej lub średniej organizacji (Cert. LCM, ITSM), albo ograniczają się do certyfikatów jednego wystawcy bez ewidencji beneficjentów, płatników i zadań (panele centrów certyfikacji). Żadne z analizowanych narzędzi nie łączy w jednej platformie rejestru certyfikatów, procesu odnowień z zaproszeniami e-mail oraz raportów dla ról personelu przy zachowaniu niskiego kosztu wdrożenia.

2.3. Technologie informatyczne stosowane w systemach zarządzania subskrypcjami

2.3.1. Architektura klient–serwer

Współczesne aplikacje webowe do zarządzania danymi są realizowane w architekturze klient–serwer. Frontend odpowiada za prezentację danych i interakcje użytkownika, backend realizuje logikę biznesową i komunikuje się z bazą danych. W projekcie CertiSub Assistant zastosowano hybrydę PHP SSR z Vue.js po stronie klienta.

2.3.2. Relacyjne bazy danych

Encje kont personelu, beneficjentów, płatników, certyfikatów, zadań odnowieniowych, szablonów wiadomości, zaproszeń i tokenów weryfikacyjnych naturalnie mapują się na tabele połączone kluczami obcymi. W projekcie wykorzystano MySQL z PDO i prepared statements, co zapewnia integralność danych i ochronę przed SQL Injection; tabela

historii zdarzeń (`events`) jest celowo wyłączona z więzów kluczy obcych, aby zachować zapis zdarzeń nawet po archiwizacji powiązanego rekordu.

2.3.3. Technologie frontendowe

Dashboard wykorzystuje Vue.js 3 do reaktywnego wyszukiwania, filtrowania i przełączania widoków bez przeładowania strony, uruchamiane bez kroku budowania — plik biblioteki jest częścią repozytorium, a nie ładowany z zewnętrznego CDN. Tailwind CSS zapewnia responsywny i spójny interfejs. Takie podejście eliminuje konieczność bundlera na etapie MVP i pozwala uruchomić aplikację bez dostępu do Internetu.

2.3.4. Uwierzytelnianie hasłem i weryfikacja adresu e-mail

W CertiSub Assistant zaimplementowano logowanie hasłem zgodnym z polityką haseł NIST SP 800-63B (minimalna długość, brak wymuszonej rotacji), z limitem prób logowania liczonym w bazie (per adres i per adres IP) oraz jednorazowymi, haszowanymi tokenami e-mail do potwierdzenia adresu i ustawienia lub zresetowania hasła. Hasła są przechowywane wyłącznie jako skrót funkcji `password_hash()` z automatycznym przeliczeniem przy zmianie kosztu algorytmu, a wysyłkę wiadomości obsługuje PHPMailer.

2.4. Systemy alertów i powiadomień o terminach płatności

Automatyczne powiadomienia o zbliżających się terminach płatności stanowią kluczowy element skutecznego zarządzania subskrypcjami. W projekcie zastosowano zadanie cron uruchamiane codziennie, które wyszukuje subskrypcje z płatnością za 3 dni i wysyła wiadomości HTML do użytkownika. Mechanizm ten uzupełnia alerty wizualne w dashboardzie (kolorowe wiersze, karty KPI, lista To-Do).

2.5. Studium wykonalności: PHP/LAMP a Java/Spring Boot

Punktem wyjścia do wyboru technologii backendu było porównanie dwóch dostępnych na uczelni i w praktyce zawodowej podejść: stosu PHP 8 na serwerze Apache z bazą MySQL (LAMP) oraz aplikacji Java ze szkieletem Spring Boot. Porównania dokonano według czterech kryteriów istotnych dla projektu realizowanego jednoosobowo w ramach pracy inżynierskiej: dostępności na standardowym hostingu, kosztu wdrożenia, złożoności uruchomienia i utrzymania oraz zgodności z kompetencjami autora zdobytymi w toku studiów.

Dostępność na hostingu

- PHP / LAMP: standardowy hosting współdzielony (Apache + PHP + MySQL).
- Java / Spring Boot: wymaga JVM i serwera aplikacyjnego; rzadziej dostępne na tanim hostingu.

Koszt wdrożenia

- PHP / LAMP: niski — stos open-source, brak opłat licencyjnych.
- Java / Spring Boot: niski licencyjnie, ale wyższe wymagania sprzętowe (JVM, więcej RAM).

Złożoność uruchomienia

- PHP / LAMP: uruchomienie skryptu przez serwer WWW, brak procesu budowania na produkcji.
- Java / Spring Boot: budowanie artefaktu (Maven/Gradle), proces JVM, konteneryzacja zalecana.

Kompetencje autora

- PHP / LAMP: PHP i MySQL z zajęć oraz praktyki zawodowej — środowisko znane.
- Java / Spring Boot: podstawy Javy z zajęć, bez doświadczenia ze Spring Boot.

Na podstawie porównania wybrano stos PHP 8 / MySQL / Apache. Decydujące okazały się: dostępność na standardowym hostingu bez dodatkowych usług (zgodnie z wymaganiem niefunkcjonalnym N3), brak kroku budowania na produkcji oraz wcześniejsze doświadczenie autora z PHP i PDO, co ograniczyło ryzyko projektu realizowanego w pojedynkę w ograniczonym czasie. Warstwę frontendową uzupełniono o Vue.js 3 uruchamiane bez bundlera (pliki w `assets/vendor`), co pozwoliło zachować prostotę wdrożenia charakterystyczną dla LAMP przy zachowaniu reaktywnego interfejsu.

2.6. Wnioski z analizy i uzasadnienie realizacji projektu

Przeprowadzona analiza problemu zarządzania certyfikatami i subskrypcjami oraz badanie dostępnych rozwiązań informatycznych pozwoliły zidentyfikować szereg ograniczeń występujących w obecnie funkcjonujących systemach. Większość narzędzi koncentruje się albo na certyfikatach jednego wystawcy, albo na ogólnej ewidencji zasobów IT bez procesu odnowień, albo wymaga ręcznego utrzymania danych bez alertów i wielożytkownikowego dostępu.

Analiza wykazała również, że dostępne rozwiązania rzadko integrują rejestr certyfikatów i subskrypcji, proces odnowień z zaproszeniami e-mail oraz raporty dla poszczególnych ról personelu w ramach jednego interfejsu. Dane są zazwyczaj rozproszone między arkuszami, portalami dostawców i skrzynkami pocztowymi, co utrudnia szybkie podejmowanie decyzji.

Na podstawie przeprowadzonych analiz uzasadniona jest realizacja aplikacji webowej CertiSub Assistant na stosie PHP/LAMP wybranym w studium wykonalności (podrozdział 2.5). Projektowany system połączy ewidencję certyfikatów, beneficjentów i płatników, proces odnowień z zadaniami ToDo i zaproszeniami e-mail, raporty dla ról personelu oraz wymianę danych. Takie podejście pozwoli zwiększyć efektywność monitorowania terminów oraz usprawnić kontrolę nad odnowieniami certyfikatów i subskrypcji organizacji.

Wnioski przedstawione w niniejszym rozdziale stanowią podstawę do opracowania założeń projektowych systemu, które zostaną przedstawione w kolejnym rozdziale pracy.

3. Założenia projektowe aplikacji

3.1. Cel i zakres projektu

Celem projektu jest firmowy rejestr certyfikatów i subskrypcji CertiSub Assistant, w którym personel organizacji (a nie użytkownicy indywidualni) wprowadza i utrzymuje dane o certyfikatach, ich użytkownikach (beneficjentach) oraz firmach płacących za certyfikaty (w opisie pracy — płatnikach), a system samodzielnie wykrywa zbliżające się terminy odnowienia, zakłada zadania, wysyła zaproszenia e-mail i rejestruje historię zdarzeń.

Zakres projektu obejmuje jedną domenę i jedną bazę danych: ewidencję certyfikatów ośmiu rodzajów (kwalifikowany podpis elektroniczny, kwalifikowana pieczęć elektroniczna, certyfikat SSL, certyfikat podpisu kodu, domena, licencja SaaS, wsparcie chmurowe oraz kategoria „inne”), użytkowników certyfikatów i firmy, proces odnowień z listą zadań ToDo, szablony wiadomości i załączniki, rejestr zaproszeń i przypomnień, raporty perspektyw (użytkownika, płatnika, administratora), wyszukiwarkę globalną, wymianę danych (import i eksport CSV, XML oraz import wiadomości EML), panel administracyjny z kontami i rolami stanowisk, a także — dodane w ostatnim etapie prac — wnioski o certyfikat przesyłane e-mailem, powiadomienia wewnętrzne między kontami i filtr firm.

Pierwotna koncepcja projektu obejmowała również osobisty moduł subskrypcji konsumenckich (streaming, muzyka, aplikacje miesięczne) dostępny każdemu zarejestrowanemu użytkownikowi. W toku prac przyjęto decyzję projektową D1: moduł prywatny został wydzielony do osobnej aplikacji („Menedżer Subskrypcji”) z własną bazą danych, kontami i interfejsem, niepowiązanej z rejestrem firmowym żadną tabelą ani odnośnikiem. Powodem były odmienne modele danych (brak beneficjenta i płatnika po stronie prywatnej), odmienny krąg odbiorców (dowolny użytkownik kontra personel z przydzieloną rolą) oraz chęć zachowania przejrzystej granicy uprawnień: operator rejestru firmowego nie powinien mieć żadnej technicznej możliwości dotarcia do prywatnych danych finansowych innego użytkownika. Niniejsza praca opisuje rejestr firmowy; aplikacja prywatna, zbudowana na tych samych zasadach jakościowych (PDO, CSRF, testy), jest wymieniona w rozdziale poświęconym kierunkom rozwoju jako powiązany, lecz niezależny projekt.

W trakcie realizacji dwie decyzje zmieniły pierwotny model danych. Po pierwsze, zamiast kwoty certyfikatu system przechowuje rabat procentowy (decyzja D10): organizacje negocjują warunki z wystawcami, a informacja o stopniu upustu jest dla personelu ważniejsza niż cennik, który zmienia się niezależnie od aplikacji. Po drugie, od Etapu 10 obowiązuje łańcuch zależności „firma → użytkownik certyfikatu → certyfikat kwalifikowany”: osoba nie istnieje bez firmy, a certyfikat kwalifikowany nie istnieje bez osoby, której dotyczy. Reguła jest wymuszana jednocześnie przez bazę danych, usługi i interfejs (podrozdział 4.4).

3.2. Wymagania funkcjonalne

Wymagania wynikają z oficjalnego opisu pracy (Załącznik A) i zostały ponumerowane

F1–F19. Poniższa lista grupuje je tematycznie; ostatnie pozycje to rozszerzenia dodane przez autora w Etapie 10. System powinien umożliwiać:

- logowanie personelu adresem e-mail i hasłem z potwierdzeniem adresu linkiem z wiadomości, zakładanie kont przez administratora (z wiadomością z linkiem do ustawienia hasła) oraz konfigurowalną rejestrację publiczną (F15),
- prowadzenie ewidencji certyfikatów: numer seryjny, wystawca, okres ważności, wymagany czas odnowienia, rabat, status płatności; dodawanie, edycję i archiwizację zamiast usuwania (F1, F5),
- prowadzenie ewidencji użytkowników certyfikatów (beneficjentów) i firm (płatników) wraz z ich powiązaniami (F2, F3, F4),
- automatyczne skanowanie certyfikatów w marginesie odnowienia i zakładanie zadań ToDo z priorytetem rosnącym wraz ze zbliżaniem się terminu (F11, F12),
- przydział zadań do personelu, zmianę statusu (do zrobienia / w toku / zrobione / porzucone), odnowienie certyfikatu z poziomu zadania i statystyki realizacji wg statusów (F12, F18),
- wysyłkę zaproszeń e-mail na podstawie szablonów z polami podstawianymi automatycznie oraz załącznikami z biblioteki plików; rejestr zaproszeń z przypomnieniami, ponowną wysyłką i oznaczaniem odpowiedzi (F13, F14),
- raporty perspektyw: karty użytkownika certyfikatu i płatnika z datami odnowienia, ścieżką realizacji i osią czasu oraz widok administratora (F6, F7, F8, F17),
- eksport danych do CSV i XML (listy, karty raportowe, harmonogram, dziennik zdarzeń) oraz import z CSV, XML i wiadomości e-mail (EML) z podglądem każdego wiersza przed zapisem (F9, F10),
- wyszukiwanie certyfikatów, osób i firm wraz z powiązaniami i podaniem powodu dopasowania (F16),
- obsługę planu kont z rolami stanowisk i zakresem danych zależnym od roli (F19),
- przyjmowanie wniosków o certyfikat przesłanych e-mailem: odczyt danych z wiadomości, uzupełnienie danych firmy z rejestru po NIP-ie, kolejka robocza operatora i zatwierdzenie jednym przyciskiem (rozszerzenie),
- powiadomienia wewnętrzne między kontami (wiadomości, prośby o uzupełnienie danych, zgłoszenia błędów, komunikaty systemowe) oraz filtr firm zawężający pracę do wybranych podmiotów (rozszerzenie),
- obsługę wielu języków interfejsu (PL jako domyślny, EN, ES, DE, UK).

3.3. Wymagania niefunkcjonalne

System powinien spełniać następujące wymagania niefunkcjonalne (N1–N6 z opisu pracy oraz wymagania własne):

- realizacja w PHP + HTML + JavaScript, zgodnie z kompetencjami autora i wynikiem studium wykonalności (podrozdział 2.5),
- baza danych zgodna z MySQL, możliwa do uruchomienia na standardowym hostingu

LAMP bez dodatkowych usług; aplikacja wdrożona w wersji demonstracyjnej na bezpłatnym hostingu,

- możliwość instalacji publicznej lub wyłącznie w sieci wewnętrznej (katalogi wewnętrzne zablokowane w konfiguracji serwera, frontend bez zależności od CDN),

- ochrona danych osobowych beneficjentów i płatników: dostęp według roli i zakresu danych, hasła przechowywane wyłącznie jako skróty, tokeny jednorazowe z terminem ważności, zapis historii zmian, w tym informacji o eksporcie danych,

- odporność na SQL Injection dzięki przygotowanym zapytaniom (PDO), ochrona przed CSRF, XSS oraz ograniczanie liczby prób logowania i wysyłek wiadomości liczone w bazie,

- responsywny, czytelny i intuicyjny interfejs działający w popularnych przeglądarkach bez konieczności instalacji wtyczek; szkielety treści zamiast pustych ekranów,

- wydajna komunikacja między frontendem a backendem przez REST API (JSON) i buforowanie kosztownych zestawień z kluczem zawierającym rolę i zakres danych,

- możliwość dalszej rozbudowy bez zmiany architektury (warstwa usług niezależna od warstwy HTTP),

- pokrycie testami jednostkowymi i integracyjnymi kluczowej logiki biznesowej (N6) oraz analiza statyczna kodu,

- niski koszt wdrożenia dzięki technologiom open-source (stos LAMP).

3.4. Role użytkowników i przypadki użycia

Opis pracy zakłada hierarchiczny plan kont z podziałem na role. W pierwszej wersji systemu zaimplementowano trzy role (administrator, menedżer, operator) ułożone w hierarchię. W Etapie 10 model rozszerzono do siedmiu profili stanowisk w firmie: okazało się, że księgowa powinna widzieć płatności całej organizacji, a informatyk — wyłącznie certyfikaty techniczne, choć żadna z tych ról nie jest „wyższa” od operatora. Każda rola ma więc własny zestaw uprawnień do akcji oraz własny zakres danych (klasa App\Rbac), a ukrycie przycisku w interfejsie nie zastępuje kontroli po stronie serwera — każda usługa wywołuje Actor::authorize().

Tabela 1. Role stanowisk, zakres danych i najważniejsze uprawnienia

Rola	Zakres danych	Najważniejsze uprawnienia
Administrator (ADMIN)	cała organizacja	wszystko: konta i role, szablony i załączniki, ustawienia, import, dziennik zdarzeń, wiadomości do wszystkich
Szef (DIRECTOR)	cała organizacja	podgląd ewidencji i archiwum, przydział opiekunów i zadań, statystyki, raporty, eksport; bez edycji
Menedżer (MANAGER)	cała organizacja	ewidencja, archiwizacja, przydziały, skaner, zaproszenia, raporty, eksport, wnioski e-mail
Księgowa (ACCOUNTANT)	cała organizacja	podgląd, zmiana płatności i rabatów, edycja danych firm, raporty, eksport
Informatyk (IT)	certyfikaty techniczne (SSL, podpis kodu, domena, SaaS, chmura, inne)	dodawanie i edycja certyfikatów technicznych, zadania, zaproszenia; bez certyfikatów kwalifikowanych
Operator (OPERATOR)	własne i przydzielone certyfikaty oraz powiązane osoby i firmy (D8)	ewidencja w swoim zakresie, zadania, zaproszenia, sprawdzanie wniosków e-mail

Rola	Zakres danych	Najważniejsze uprawnienia
Pracownik (EMPLOYEE)	wyłącznie własne certyfikaty i ich firmy	tylko odczyt, własne zadania, raporty, powiadomienia

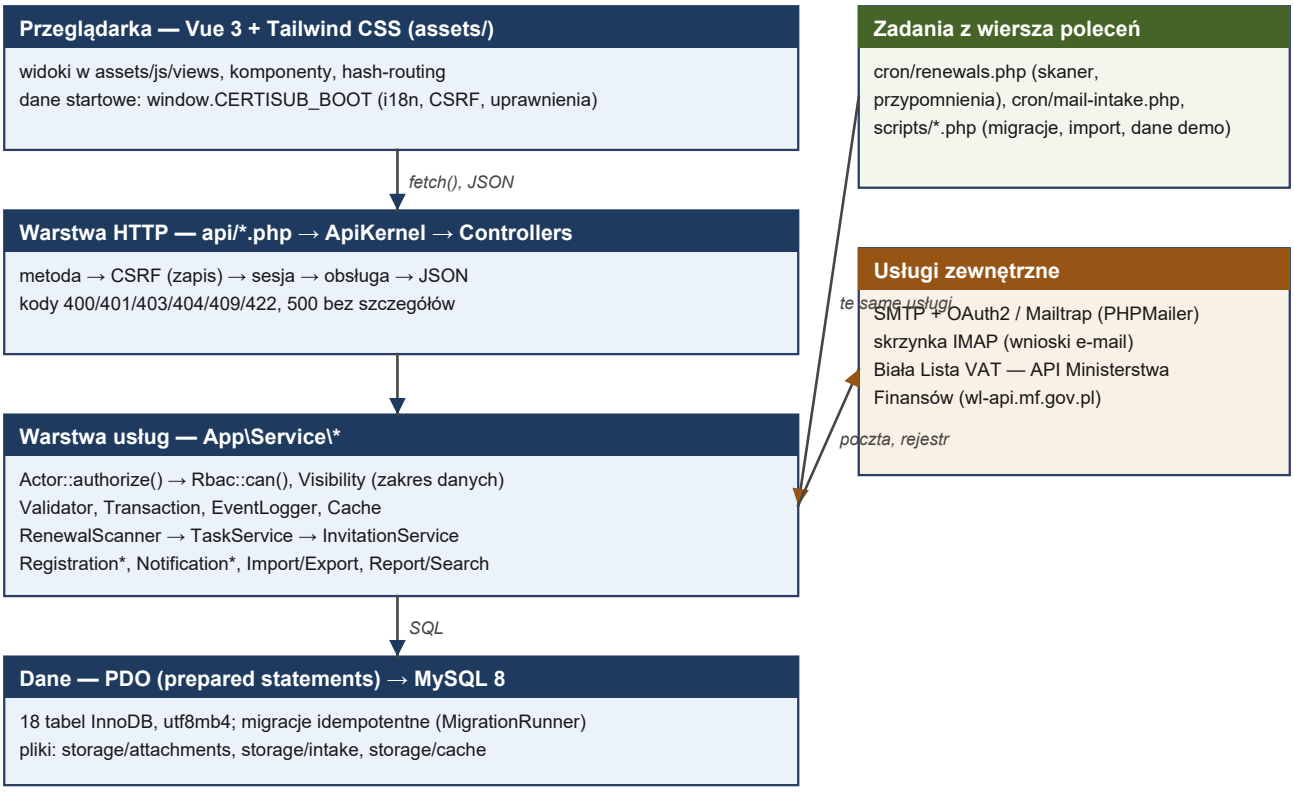
Zakres operatora wynika z zasady najmniejszych uprawnień (decyzja D8): operator widzi certyfikaty, których jest opiekunem lub ma przydzielone zadanie, oraz osoby i firmy, które sam wprowadził albo które są powiązane z jego certyfikatami. Pracownik (EMPLOYEE) może mieć konto powiązane z własnym rekordem użytkownika certyfikatu i wtedy widzi również certyfikaty, w których jest beneficjentem. Pełna macierz uprawnień znajduje się w załączniku B.

Typowe przypadki użycia:

- UC1 — Logowanie: użytkownik podaje adres e-mail i hasło; po pięciu nieudanych próbach w ciągu 15 minut logowanie jest blokowane; konto z niepotwierdzonym adresem nie zaloguje się.
- UC2 — Rejestracja certyfikatu (operator, menedżer): wybór firmy i użytkownika certyfikatu, wpisanie numeru seryjnego, wystawcy, dat i rabatu; formularz podświetla błędy przy polach.
- UC3 — Obsługa zadania odnowienia (operator): zadanie założone przez skaner trafia na listę ToDo; operator rozpoczyna pracę, wysyła zaproszenie z szablonu i po odnowieniu zamyka zadanie z nową datą ważności — stary certyfikat trafia do archiwum.
- UC4 — Przegląd firmy (księgowa, szef): karta firmy z harmonogramem wygaśnięć i rabatami; eksport zestawienia do CSV.
- UC5 — Wniosek z e-maila (operator): wiadomość z danymi wnioskodawcy zostaje zamieniona na formularz z danymi firmy z rejestru VAT; operator sprawdza i zatwierdza wniosek.
- UC6 — Administracja (administrator): założenie konta z rolą, przekazanie rekordów odchodzącego pracownika, zmiana progów procesu odnowień, przegląd dziennika zdarzeń.

3.5. Architektura ogólna systemu

Aplikacja jest zrealizowana w architekturze klient–serwer z podziałem na cztery warstwy: prezentacji (Vue 3 bez kroku budowania, Tailwind CSS), HTTP (jądro API mapujące metodę żądania, token CSRF i sesję na odpowiedź JSON ze statusem błędu), usług domenowych (klasy w przestrzeni App\Service, każda z walidacją, kontrolą uprawnień przez Rbac::can(), zakresem danych i zapisem zdarzeń do historii) oraz dostępu do danych (MySQL przez PDO z przygotowanymi zapytaniami). Rysunek poniżej przedstawia warstwy i miejsca integracji z usługami zewnętrznymi.



Rysunek 1. Architektura warstwowa aplikacji CertiSub Assistant

Warstwa usług nie odwołuje się bezpośrednio do sesji HTTP — przyjmuje połączenie PDO i obiekt aktora (Actor) jako parametry, dzięki czemu ta sama logika jest wykorzystywana zarówno przez kontrolery API, jak i przez skrypty uruchamiane z linii poleceń (skaner odnowień, odbiór poczty, import danych), bez duplikowania reguł biznesowych. Taki podział to odmiana wzorca „warstwy usług" opisanego m.in. przez M. Fowlera [9].

Centralnym mechanizmem systemu jest proces odnowień: cykliczne zadanie cron/renewals.php wyszukuje certyfikaty w marginesie odnowienia i zakłada zadanie ToDo, zadanie trafia na listę personelu z priorytetem zależnym od liczby dni do wygaśnięcia, obsługa zadania może zakończyć się wysyłką zaproszenia e-mail (z szablonu i załączników) do beneficjenta lub płatnika, a finalnie — odnowieniem certyfikatu (zapisaniem nowego certyfikatu powiązanego z poprzednim, który trafia do archiwum) albo świadomym porzuceniem zadania. Każdy krok zapisuje zdarzenie w tabeli events, na podstawie której budowane są oś czasu i raporty perspektyw.

System stosuje dodatkowo: warstwę pamięci podręcznej dla kosztownych zestawień (agregaty KPI i statystyki zadań) z kluczem zawierającym rolę, konto i filtr firm oraz unieważnianiem przy każdym zapisie; moduł wymiany danych (App\Exchange) z jednolitym opisem kolumn i normalizacją wartości, współdzielonym przez eksport i import; moduł odbioru wniosków (App\Intake) z własnym klientem IMAP oraz klientami otwartych rejestrów firm (Biała Lista, VIES, KRS); mechanizm i18n z domyślnym językiem polskim oraz zestaw testów jednostkowych i integracyjnych (PHPUnit) i statyczną analizę kodu (PHPStan) uruchamiane przy każdej zmianie.

4. Projekt bazy danych

4.1. Model pojęciowy danych

Model danych obejmuje sześć grup encji. Centralną encją jest certyfikat, powiązany jednocześnie z kontem opiekuna (personel), użytkownikiem certyfikatu (beneficjentem) i firmą-płatnikiem, a poprzez pole previous_certificate_id — także z poprzednim certyfikatem w łańcuchu kolejnych odnowień. Baza liczy 18 tabel (17 tabel domenowych i tabelę schema_migrations).

Główne encje i ich rola w systemie:

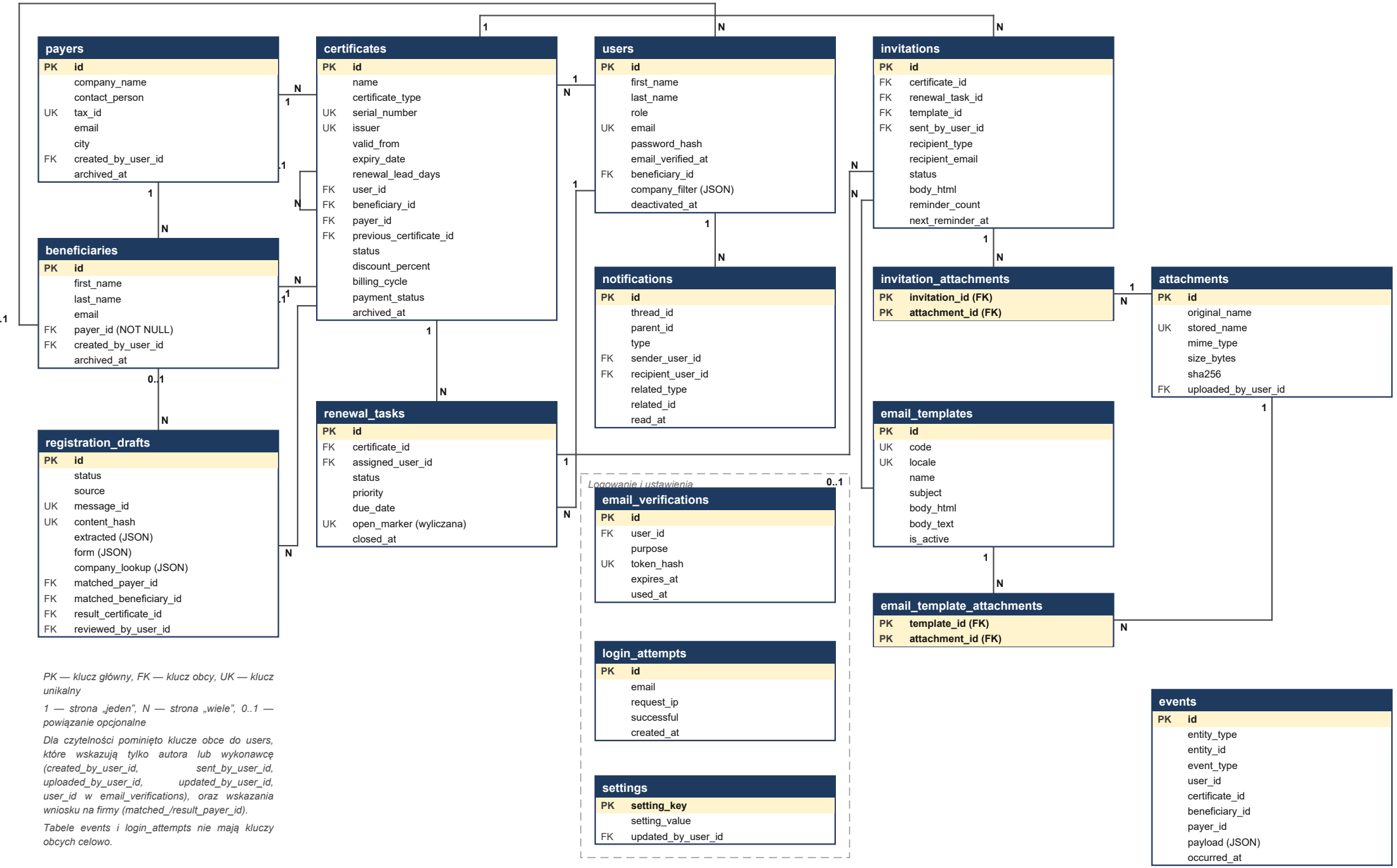
- Konto personelu (users) — pracownik organizacji z jedną z siedmiu ról; opiekun certyfikatów i wykonawca zadań; konto może wskazywać swoją osobę (beneficiary_id) i przechowuje filtr firm,
- Firma / płatnik (payers) — podmiot rozliczający certyfikaty, z unikalnym NIP-em (tax_id),
- Użytkownik certyfikatu / beneficjent (beneficiaries) — osoba, której dotyczy certyfikat; zawsze przypisana do firmy,
- Certyfikat (certificates) — pozycja podlegająca odnowieniu: typ, numer seryjny, wystawca, okres ważności, rabat, status płatności,
- Zadanie odnowieniowe (renewal_tasks) — wpis listy ToDo założony przez skaner albo ręcznie,
- Szablon wiadomości (email_templates), załącznik (attachments) i zaproszenie (invitations) wraz z tabelami łączącymi — treści i pliki wielokrotnego użytku oraz rejestr wysyłek,
- Wniosek z e-maila (registration_drafts) i powiadomienie (notifications) — kolejka robocza operatora i komunikacja wewnętrzna,
- Zdarzenie (events) i ustawienie (settings) — historia operacji i administracyjne nadpisania progów procesu odnowień,
- Tabele obsługujące logowanie: email_verifications, login_attempts oraz login_otps (historia logowań kodem sprzed Etapu 9).

Relacje między głównymi encjami: firma 1–N użytkownik certyfikatu (relacja obowiązkowa), użytkownik certyfikatu 1–N certyfikat, firma 1–N certyfikat (certyfikat może być opłacany przez inną firmę niż domyślna firma beneficjenta, np. po zmianie pracodawcy), certyfikat 1–N zadanie odnowieniowe (co najwyżej jedno zadanie otwarte naraz), zadanie 1–N zaproszenie, szablon N–N załącznik, a wszystkie encje biznesowe łączy relacja 1–N ze zdarzeniami historii.

4.2. Diagram ERD

Rysunek na następnej stronie przedstawia pełny model związków encji (ERD) bazy assistent_subscriptions: tabele, klucze główne, obce i unikalne oraz kardynalność relacji opisanych w podrozdziale 4.1. Dla czytelności pominięto klucze obce do tabeli users, które wskazują wyłącznie autora lub wykonawcę zapisu; pełną listę zawierają plik database/schema.sql oraz dokument docs/BAZA_DANYCH.md.

Rysunek 2. Diagram ERD bazy danych assistent_subscriptions (stan po Etapie 10)



PK — klucz główny, FK — klucz obcy, UK — klucz unikalny

1 — strona „jeden", N — strona „wiele", 0..1 — powiązanie opcjonalne

Dla czytelności pominięto klucze obce do users, które wskazują tylko autora lub wykonawcę (created_by_user_id, sent_by_user_id, uploaded_by_user_id, updated_by_user_id, user_id w email_verifications), oraz wskazania wniosku na firmy (matched_/result_payer_id).

Tabele events i login_attempts nie mają kluczy obcych celowo.

4.3. Struktura bazy danych i relacje między encjami

Baza działa na silniku InnoDB ze zestawem znaków utf8mb4 (pełne wsparcie Unicode, w tym polskich znaków diakrytycznych). Schemat jest utrzymywany w dwóch równoważnych postaciach: jako plik database/schema.sql (świeża instalacja) oraz jako ciąg idempotentnych migracji (aktualizacja istniejących baz); oba źródła dają identyczny kształt bazy — 190 kolumn, 87 pozycji indeksów, 32 klucze obce i 2 ograniczenia CHECK — co sprawdzono porównaniem information_schema.

Tabela 2. Tabele bazy danych i ich przeznaczenie

Tabela	Przeznaczenie
users	konta personelu: rola, skrót hasła, potwierdzenie adresu, filtr firm, powiązanie z osobą
payers	firmy (płatnicy): nazwa, NIP, adres, osoba kontaktowa
beneficiaries	użytkownicy certyfikatów: dane osobowe, obowiązkowa firma
certificates	certyfikaty i usługi: typ, numer seryjny, wystawca, daty, rabat, płatność
renewal_tasks	lista ToDo: zadania odnowień ze statusem, priorytetem i przydziałem
email_templates	szablony wiadomości (kod + język)
attachments, email_template_attachments, invitation_attachments	biblioteka plików i jej powiązania z szablonami oraz zaproszeniami
invitations	rejestr zaproszeń i przypomnień z treścią z chwili wysyłki
registration_drafts	wnioski o certyfikat odczytane z e-maili — kolejka robocza operatora
notifications	powiadomienia wewnętrzne między kontami (wątki, prośby, komunikaty systemowe)
events	historia zdarzeń (oś czasu, dziennik administratora), bez kluczy obcych
settings	administracyjne nadpisania progów procesu odnowień
email_verifications, login_attempts, login_otps	tokeny z e-maili, limit prób logowania, historia kodów OTP
schema_migrations	lista zastosowanych migracji

users

Konta personelu: first_name, last_name, role (ENUM siedmiu ról), email (unikalny), password_hash (skrót hasła, NULL dla konta bez ustawionego jeszcze hasła), email_verified_at (NULL blokuje logowanie do czasu potwierdzenia adresu), last_login_at, deactivated_at (wyłączenie konta bez usuwania historii), beneficiary_id (konto pracownika wskazuje swoją osobę; klucz SET NULL) oraz company_filter (JSON z filtrem firm wybranym w panelu).

payers i beneficiaries

Firma: nazwa, osoba kontaktowa, NIP (tax_id, unikalny także w archiwum), dane adresowe, created_by_user_id (kto wprowadził rekord — podstawa zakresu danych operatora), archived_at. Użytkownik certyfikatu: imię, nazwisko, kontakt, notatki oraz payer_id — kolumna NOT NULL z kluczem obcym ON DELETE RESTRICT, więc osoba nie istnieje bez firmy, a firmy z osobami nie da się usunąć.

certificates

Najbardziej rozbudowana tabela: certificate_type (ENUM ośmiu rodzajów), serial_number i issuer z unikalnym kluczem złożonym (issuer, serial_number) — ten sam numer seryjny może wystąpić u dwóch różnych wystawców, ale nie dwukrotnie u tego samego;

valid_from, expiry_date, renewal_lead_days (indywidualny margines odnowienia), klucze obce do users (opiekun), beneficiaries, payers oraz previous_certificate_id (łańcuch odnowień, ON DELETE SET NULL), a także status, payment_status, billing_cycle, last_payment_date, auto_renew i archived_at. Rabat przechowuje kolumna discount_percent DECIMAL(5,2) z ograniczeniem CHECK od 0 do 100; interfejs pokazuje go z minusem (np. -5%). Ograniczenie chk_certificates_qualified_user wymaga użytkownika certyfikatu dla typów kwalifikowanych.

renewal_tasks

Zadanie ToDo: certificate_id, assigned_user_id, status (todo / in_progress / done / abandoned), priority (expired / critical / warning / ok), due_date, resolution_note. Kolumna wyliczana open_marker (1 dla zadań otwartych, NULL dla zamkniętych) w parze z unikalnym kluczem (certificate_id, open_marker) wymusza na poziomie bazy regułę „co najwyżej jedno otwarte zadanie na certyfikat” — MySQL traktuje NULL jako wartość niepowtarzalną, więc zamknięte zadania nie kolidują ze sobą.

email_templates, attachments, invitations

Szablon wiadomości ma unikalną parę (code, locale). Załączniki (attachments) przechowują nazwę oryginalną, losową nazwę pliku na dysku (storage/attachments), typ MIME, rozmiar i skrót SHA-256; ich powiązania z szablonami i zaproszeniami to tabele łączące N–N. Zaproszenie zapisuje treść z chwili wysyłki (niezależnie od późniejszych zmian szablonu), odbiorcę, status (queued / sent / failed / responded / closed), licznik i termin następnego przypomnienia.

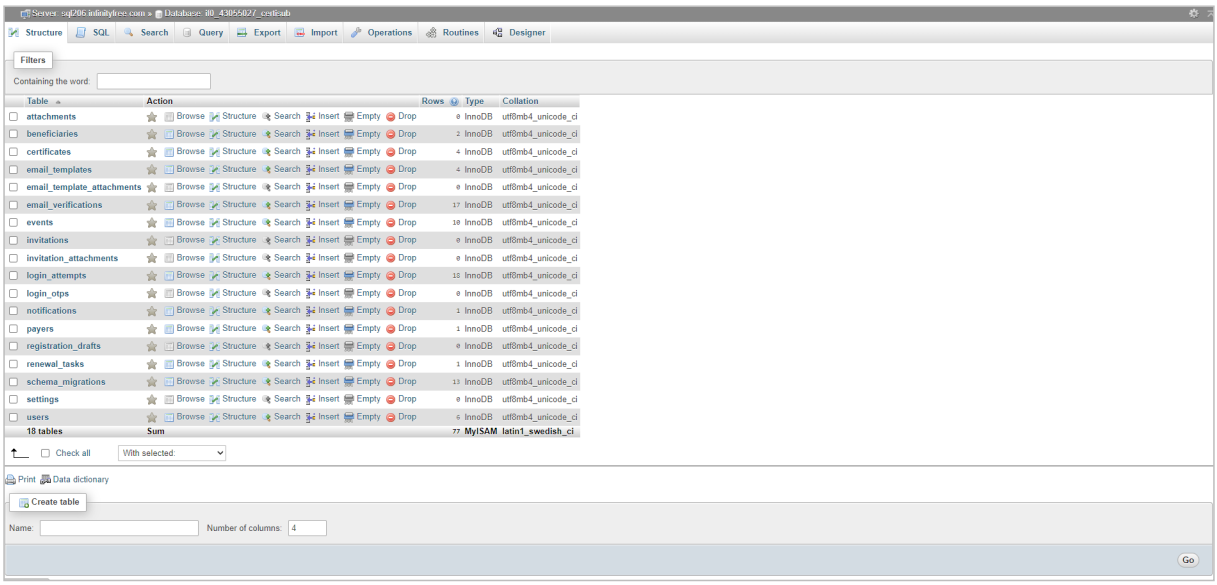
registration_drafts i notifications

Wniosek z e-maila przechowuje status (pending / approved / rejected), źródło (upload, imap, webhook, cli), unikalne message_id i content_hash (ochrona przed podwójnym wczytaniem tej samej wiadomości), nadawcę, treść oraz cztery dokumenty JSON: extracted (co odczytano), form (formularz edytowany przez operatora), company_lookup (odpowiedź rejestru VAT) i warnings (wątpliwości). Kolumny matched_* wskazują rekordy już istniejące, result_* — rekordy utworzone przy zatwierdzeniu. Powiadomienie to jeden wiersz na odbiorcę z polami thread_id i parent_id (wątki) oraz related_type / related_id (rekord, którego dotyczy wiadomość).

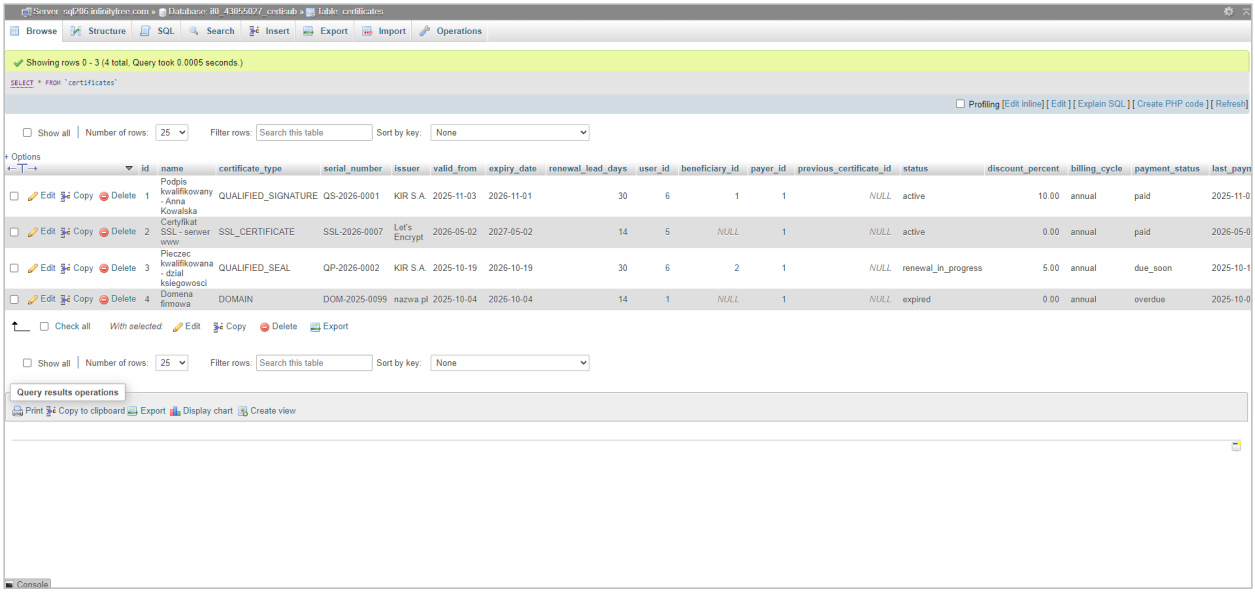
events

Historia zdarzeń: typ i identyfikator encji, typ zdarzenia, konto wykonujące, kolumny kontekstu (certyfikat, użytkownik certyfikatu, firma), dane w formacie JSON oraz znacznik czasu. Tabela celowo nie ma kluczy obcych, dzięki czemu zdarzenie pozostaje czytelne nawet po archiwizacji lub zmianie powiązanego rekordu.

Poniższe zrzuty przedstawiają bazę wdrożoną w wersji demonstracyjnej na bezpłatnym hostingu, oglądaną w phpMyAdmin: listę 18 tabel oraz zawartość tabeli certyfikatów.



Rysunek 3. Lista tabel bazy w phpMyAdmin (wersja demonstracyjna)



Rysunek 4. Zawartość tabeli certificates w phpMyAdmin (dane demonstracyjne)

4.4. Integralność i bezpieczeństwo danych

Integralność i bezpieczeństwo danych zapewnia kilka mechanizmów działających razem:

- wszystkie zapytania SQL wykonywane przez PDO z przygotowanymi zapytaniami, przy wyłączonej emulacji (PDO::ATTR_EMULATE_PREPARES = false) — parametry nigdy nie są sklejane z treścią zapytania, co eliminuje SQL Injection [5],

- łańcuch „firma → użytkownik → certyfikat kwalifikowany” wymuszony trzema warstwami: baza (NOT NULL, CHECK, klucze RESTRICT), usługi (błędy przy polach) i interfejs (pola wymagane); baza jest ostatnią linią obrony — wyjątek SQL zamiast cichego zapisu niespójnych danych,

- klucze obce dobrane do semantyki relacji: RESTRICT tam, gdzie usunięcie rekordu nadrzędnego zepsułoby historię (np. certyfikat → firma), SET NULL tam, gdzie powiązanie jest pomocnicze (np. zadanie → opiekun po wyłączeniu konta), CASCADE

dla danych ściśle zależnych od rekordu nadrzędnego (np. tokeny konta, załączniki szablonu),

- archiwizacja zamiast usuwania (archived_at) dla certyfikatów, użytkowników certyfikatów i firm — rekord i jego historia zostają w bazie, a zapytania „bieżące” filtrują archived_at IS NULL; rekord archiwalny jest tylko do odczytu,

- unikalne klucze wymuszające reguły biznesowe w bazie, nie tylko w kodzie: adres e-mail konta, NIP firmy, para wystawca + numer seryjny, co najwyżej jedno otwarte zadanie na certyfikat, Message-ID i skrót treści wniosku e-mail,

- hasła przechowywane wyłącznie jako skrót password_hash(), tokeny weryfikacji adresu i ustawienia hasła — jako skrót SHA-256 z terminem ważności i znacznikiem zużycia [4],

- limit prób logowania i wysyłek wiadomości liczony w bazie (login_attempts, email_verifications) per adres e-mail i per adres IP, co uniemożliwia obejście przez wyczyszczenie ciasteczek,

- migracje schematu jako idempotentne klasy (App\MigrationRunner, classes/Migrations/) sprawdzające stan bazy przed każdym krokiem (SchemaInspector); przed migracją zalecany jest zrzut bazy (mysqldump), bo migracja rabatu usuwa dotychczasową kolumnę z kwotą,

- przypisanie danych osobowych do ról: zakres danych liczony z roli (Visibility), nie zapisany w tabelach; eksport danych zapisuje zdarzenie z autorem i czasem (rozliczalność w rozumieniu RODO [2]).

5. Projekt i implementacja aplikacji webowej

5.1. Wykorzystane technologie i narzędzia

Zestaw technologii wynika ze studium wykonalności (podrozdział 2.5) i z wymagania, by aplikacja działała na standardowym hostingu LAMP. Poniższa tabela zawiera wersje używane w projekcie oraz ich rolę.

Tabela 3. Technologie i narzędzia wykorzystane w projekcie

Warstwa	Technologia (wersja)	Zastosowanie
Backend	PHP 8.3 (wymagane ≥ 8.1), PDO	logika biznesowa, API JSON, skrypty CLI; strict_types, klasy final, przestrzeń nazw App
Baza danych	MySQL 8.4 (InnoDB, utf8mb4)	ewidencja, historia zdarzeń, kolejki; CHECK, kolumny wyliczane, JSON
Frontend	Vue 3.5 (bez kroku budowania), Tailwind CSS 3.4	reaktywny panel; arkusz stylów budowany lokalnie i zapisany w repozytorium
Poczta	PHPMailer 6.9; SMTP + OAuth2 (XOAUTH2), Mailtrap, własny klient IMAP	wiadomości z linkami, zaproszenia, przypomnienia, odbiór wniosków
Integracje	API Białej Listy podatników VAT (Ministerstwo Finansów)	uzupełnianie danych firmy po NIP-ie
Testy i jakość	PHPUnit 11, PHPStan 2 (poziom 5), PHP-CS-Fixer 3	testy jednostkowe i integracyjne, analiza statyczna, styl PSR-12
Środowisko	Windows, Laragon (Apache, PHP, MySQL); Composer; npm (tylko do stylów); Git, GitHub	praca lokalna, kontrola wersji
Wdrożenie demonstracyjne	bezpłatny hosting InfinityFree (LAMP), phpMyAdmin	wersja do oglądania przez promotora, bez dostępu SSH

Kod aplikacji liczy około 18,7 tys. wierszy PHP w 108 plikach klas, 8,2 tys. wierszy JavaScript w 19 plikach interfejsu oraz 8 tys. wierszy testów w 52 plikach; plik tłumaczeń polskich zawiera ponad 1200 kluczy. Zależności Composera ograniczono do jednej biblioteki produkcyjnej (PHPMailer); pozostałe elementy — klient IMAP, parser wiadomości MIME, klient OAuth2, klient rejestru VAT — napisano samodzielnie, aby aplikacja działała na hostingu bez rozszerzeń takich jak mailparse czy imap.

5.2. Implementacja warstwy backendowej (PHP)

Punkt wejścia każdego żądania ładuje plik bootstrap.php, który ustawia strefę czasową Europe/Warsaw (PHP i MySQL muszą liczyć daty w tej samej strefie — w przeciwnym razie dni do wygaśnięcia liczone w PHP rozjeżdżały się o jeden dzień z licznikami opartymi na CURDATE()), rejestruje autoloader klas z przestrzeni App, uruchamia sesję i tłumaczenia. Połączenie z bazą tworzy klasa Database z trybem wyjątków, domyślnym pobieraniem asocjacyjnym i wyłączoną emulacją zapytań przygotowanych.

Logika biznesowa mieszka w usługach (classes/Service). Usługa przyjmuje połączenie PDO i obiekt Actor opisujący zalogowane konto (id, rola, powiązana osoba, filtr firm), nigdy nie czyta sesji. Każda metoda zaczyna się od Actor::authorize('uprawnienie'), co wywołuje Rbac::can() (listing 1); następnie walidator sprawdza dane, a zakres rekordów ogranicza klasa Visibility, która zamienia rolę w warunek SQL z unikalnie nazwanymi parametrami (przy wyłączonej emulacji PDO ta sama nazwa parametru nie może wystąpić w zapytaniu dwa razy). Zapisy wielotabelowe wykonuje Transaction::run, a

każda zmiana biznesowa jest rejestrowana przez EventLogger z identyfikatorami kontekstu (certyfikat, osoba, firma), z których powstają osie czasu.

Listing 1. Rbac::can() — nieznana rola albo uprawnienie oznacza brak dostępu

```
public static function can(?string $role, string $permission): bool
{
    $profile = self::PROFILES[self::normalize($role)] ?? null;
    if ($profile === null || !in_array($permission, self::ALL, true)) {
        return false;
    }

    return $profile['can'] === null || in_array($permission, $profile['can'], true);
}
```

Warstwa HTTP jest cienka: każdy plik w katalogu api/ to jedna linia uruchamiająca kontroler przez ApiKernel::run(). Jądro (listing 2) wymusza stałą kolejność kontroli — metoda żądania, token CSRF dla zapisów, sesja i aktywne konto, poprawność JSON — a następnie mapuje wyjątki usług na kody HTTP (400, 403, 404, 409, 422) wraz z błędami przypisanymi do pól formularza. Nieoczekiwany wyjątek kończy się kodem 500 bez ujawniania szczegółów, a szczegóły trafiają do dziennika błędów serwera. Dzięki temu logika jest testowalna bez serwera WWW, a kolejność kontroli — sprawdzona jednym testem jednostkowym.

Listing 2. ApiKernel::handle() — kolejność kontroli żądania (skrócony)

```
if (!in_array($request->method, $methods, true)) {
    return self::error(405, __('api.error.method'));
}
if (!$request->isRead() && !($this->csrfValidator)($request->csrfToken)) {
    return self::error(403, __('api.error.csrf'));
}
$user = ($this->userResolver)();
if ($user === false) {
    return self::error(401, __('api.error.unauthorized'));
}
try {
    $result = $handler($request, Actor::fromUser($user));
    return Response::json(200, ['success' => true] + $result);
} catch (ServiceException $e) {
    return new Response($e->httpStatus(), [...]);   // 400/403/404/409/422
} catch (Throwable $e) {
    error_log(...);
    return self::error(500, __('api.error.server'));
}
```

Pozostałe elementy backendu: klasa Cache zapisuje agregaty pulpitu w plikach JSON z czasem życia pięć minut, a EventLogger przy każdym zapisie podbija wersję, co unieważnia wszystkie wpisy; MigrationRunner stosuje migracje z katalogu classes/Migrations, sprawdzając stan bazy przez SchemaInspector, więc ponowne uruchomienie niczego nie psuje; Translator ładuje tłumaczenia z katalogu lang (polski domyślnie, angielski, a hiszpański, niemiecki i ukraiński korzystają z angielskich kluczy zastępczych). Skrypty CLI (scripts/, cron/) zaczynają się od sprawdzenia PHP_SAPI, więc przeglądarka dostaje 403.

5.3. Implementacja warstwy frontendowej (Vue.js, Tailwind CSS)

Panel jest aplikacją jednostronicową napisaną w Vue 3 bez kroku budowania: plik biblioteki znajduje się w assets/vendor, a kod interfejsu w assets/js (core.js — klient API,

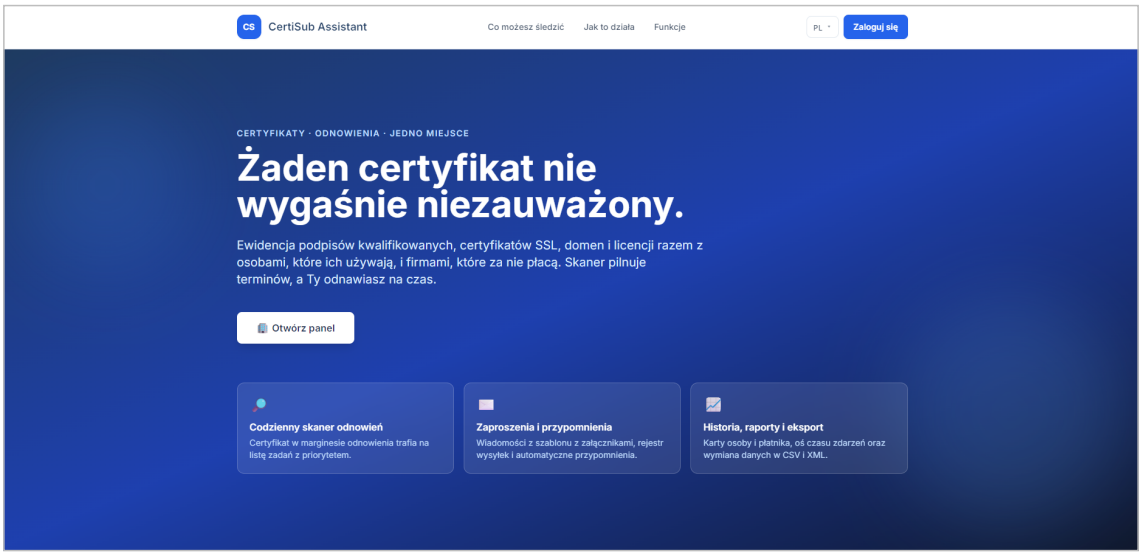
store, powiadomienia i stos okien; components.js — wspólne komponenty; views/ — po jednym pliku na widok; app.js — montowanie i routing). Widoki są adresowane znacznikiem hash (np. dashboard.php#/payers/22), więc odświeżenie strony zostawia użytkownika w tym samym miejscu. Strona dashboard.php sprawdza sesję po stronie serwera i wstrzykuje obiekt window.CERTISUB_BOOT: tłumaczenia, token CSRF, dane konta, listę uprawnień i adresy punktów API. Lista uprawnień służy wyłącznie do ukrywania przycisków — dostępu pilnuje serwer.

Do stylowania użyto Tailwind CSS 3.4. Arkusz jest budowany poleceniem npm run css z pliku assets/css/app.src.css (klasy komponentów) i zapisany jako assets/css/app.css w repozytorium, dzięki czemu aplikacja uruchamia się bez Node.js i bez dostępu do Internetu — wymagało tego wymaganie niefunkcjonalne N3 (instalacja w sieci wewnętrznej); także krój Inter jest dostarczany lokalnie. Zasoby mają wersję w adresie (parametr ?v=czas zmiany pliku), więc serwer może je oddawać z nagłówkiem Cache-Control: immutable na rok, a strony PHP i API — zawsze z no-store.

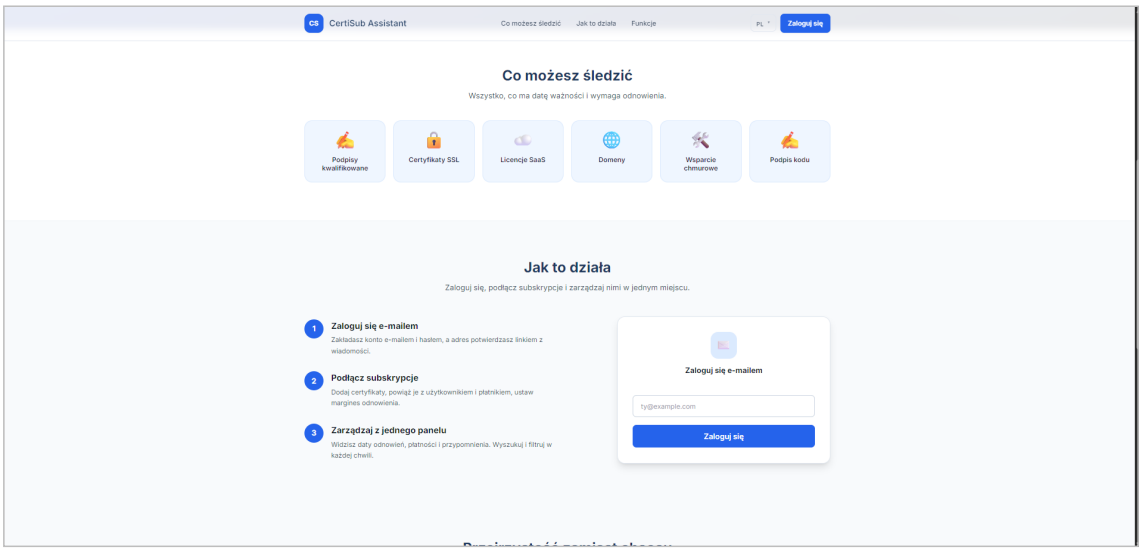
Doświadczenie ładowania poprawiają szkielety treści: układ strony (menu, pasek wyszukiwania, kafelki, tabela) jest częścią HTML, więc użytkownik nie widzi pustego ekranu przed zamontowaniem Vue; wiersze tabel, kafelki KPI, panele szczegółów i formularze mają własne szkielety o tych samych wymiarach co docelowa treść, a animacja jest wyłączana, gdy system ma ustawione ograniczenie ruchu (prefers-reduced-motion). Elementy szkieletu mają atrybuty ARIA, dzięki czemu czytnik ekranu zapowiada „Ładowanie…” zamiast czytać puste pola.

5.4. Implementacja dashboardu i formularzy

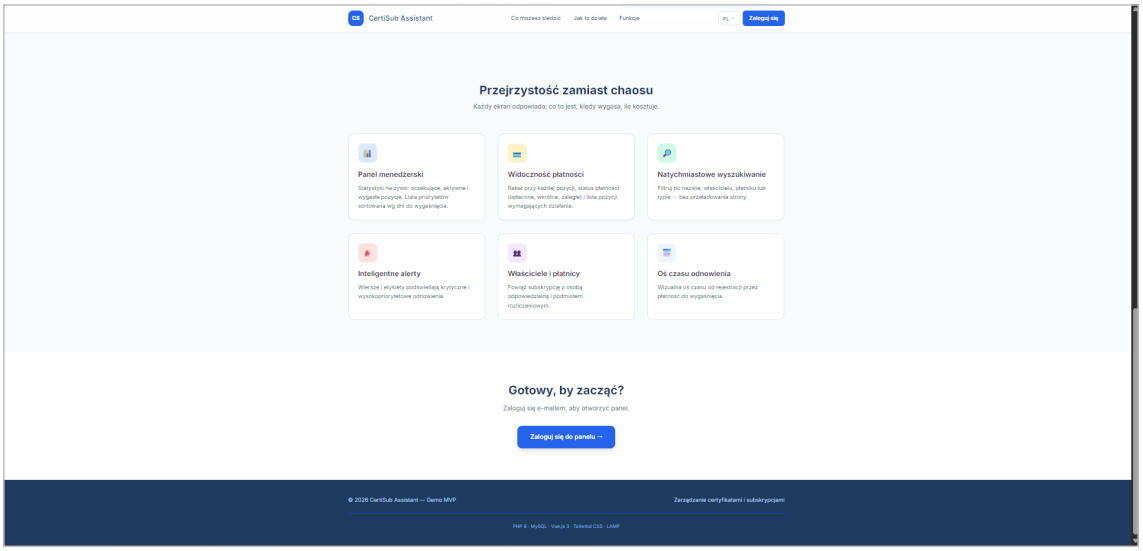
Pierwszym elementem, który widzi odwiedzający, jest strona główna (trzy rysunki poniżej) — krótko opisuje rodzaje pozycji do śledzenia (podpisy kwalifikowane, certyfikaty SSL, licencje SaaS, domeny, wsparcie chmurowe, podpis kodu), sposób działania oraz funkcje systemu i prowadzi do logowania.



Rysunek 5. Strona główna aplikacji — nagłówek z opisem funkcji

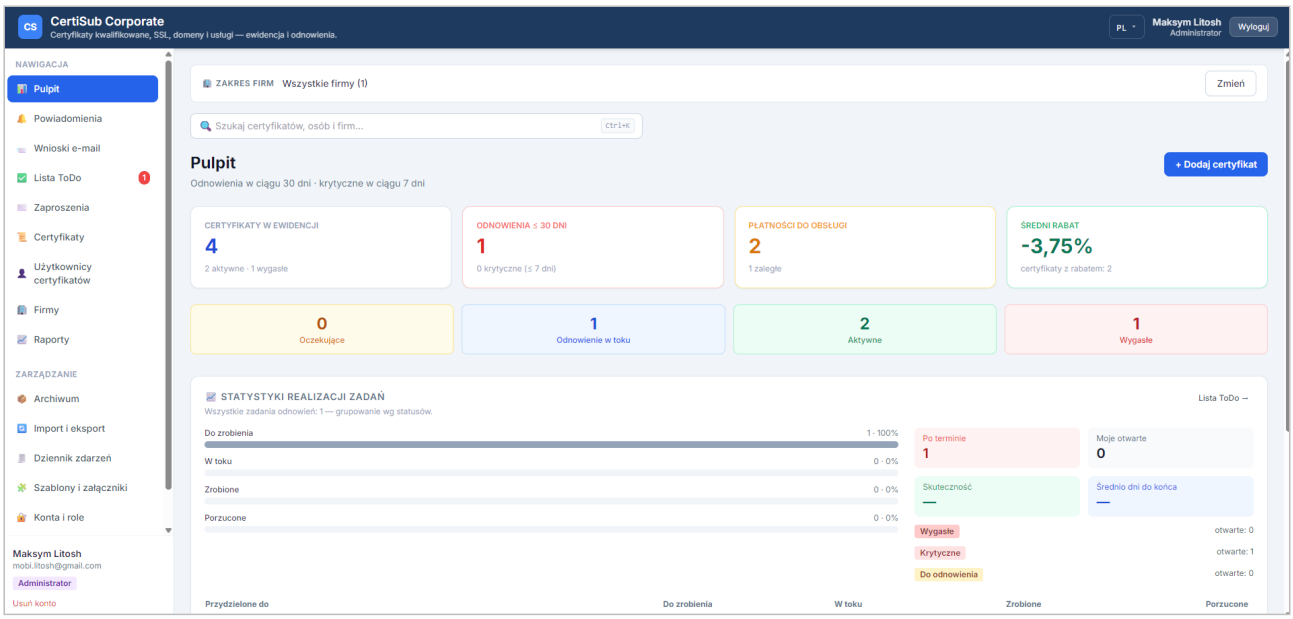


Rysunek 6. Strona główna — rodzaje śledzonych pozycji i opis działania



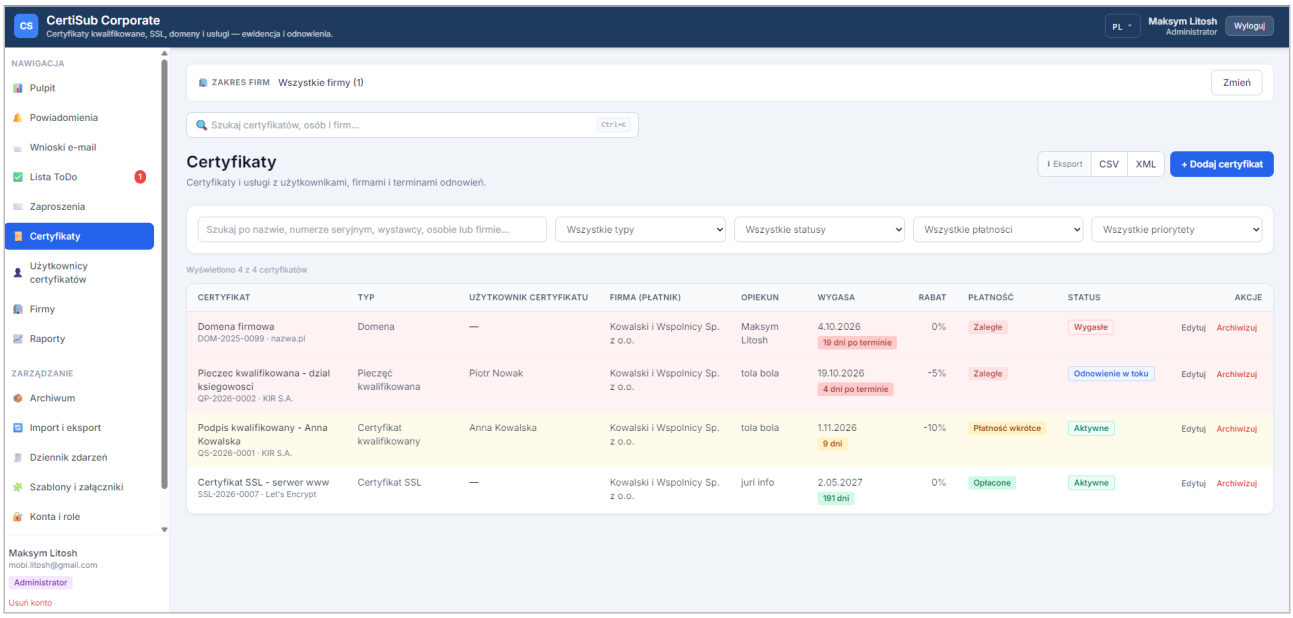
Rysunek 7. Strona główna — przegląd funkcji i zaproszenie do logowania

Po zalogowaniu użytkownik trafia na pulpit (rysunek poniżej). Układ tworzą: nagłówek z nazwą aplikacji, wyborem języka i kontem, menu boczne zależne od uprawnień roli, pasek „Zakres firm” (filtr firm), globalna wyszukiwarka (Ctrl+K) i treść widoku. Cztery kafelki KPI pokazują liczbę certyfikatów w ewidencji, odnowienia w ciągu 30 dni (w tym krytyczne w ciągu 7), płatności do obsługi oraz średni rabat; pod nimi znajdują się liczniki statusów (oczekujące, odnowienie w toku, aktywne, wygasłe), lista priorytetowych odnowień i panel statystyk realizacji zadań z podziałem na statusy i osoby. Wskaźniki są liczone raz na pięć minut dla każdej roli i konta i odświeżają się natychmiast po zapisie danych.

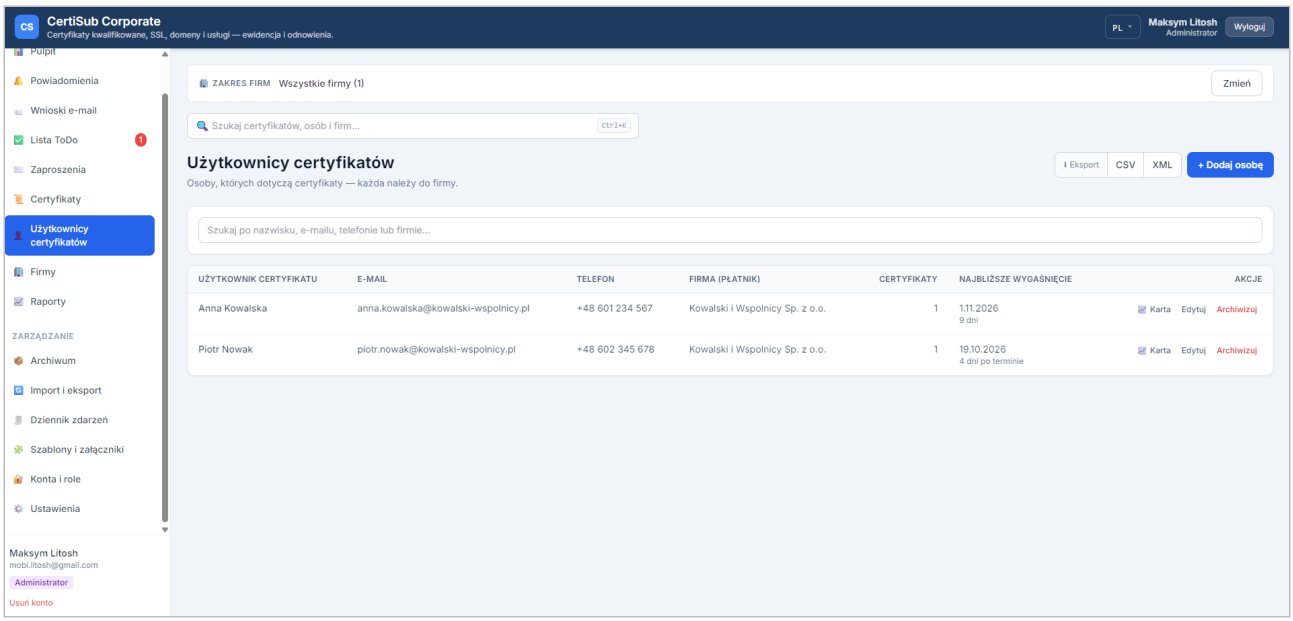


Rysunek 8. Pulpit administratora: wskaźniki KPI, liczniki statusów i statystyki zadań

Listy rekordów (rysunki poniżej) mają jednolity układ: nagłówek z przyciskami eksportu CSV i XML oraz akcją dodawania, wyszukiwarkę i filtry (typ, status, płatność, priorytet), tabelę z kolorami wierszy zależnymi od pilności i panel szczegółów otwierany kliknięciem wiersza. Lista certyfikatów pokazuje m.in. rabat (np. -5%), status płatności i liczbę dni do wygaśnięcia.

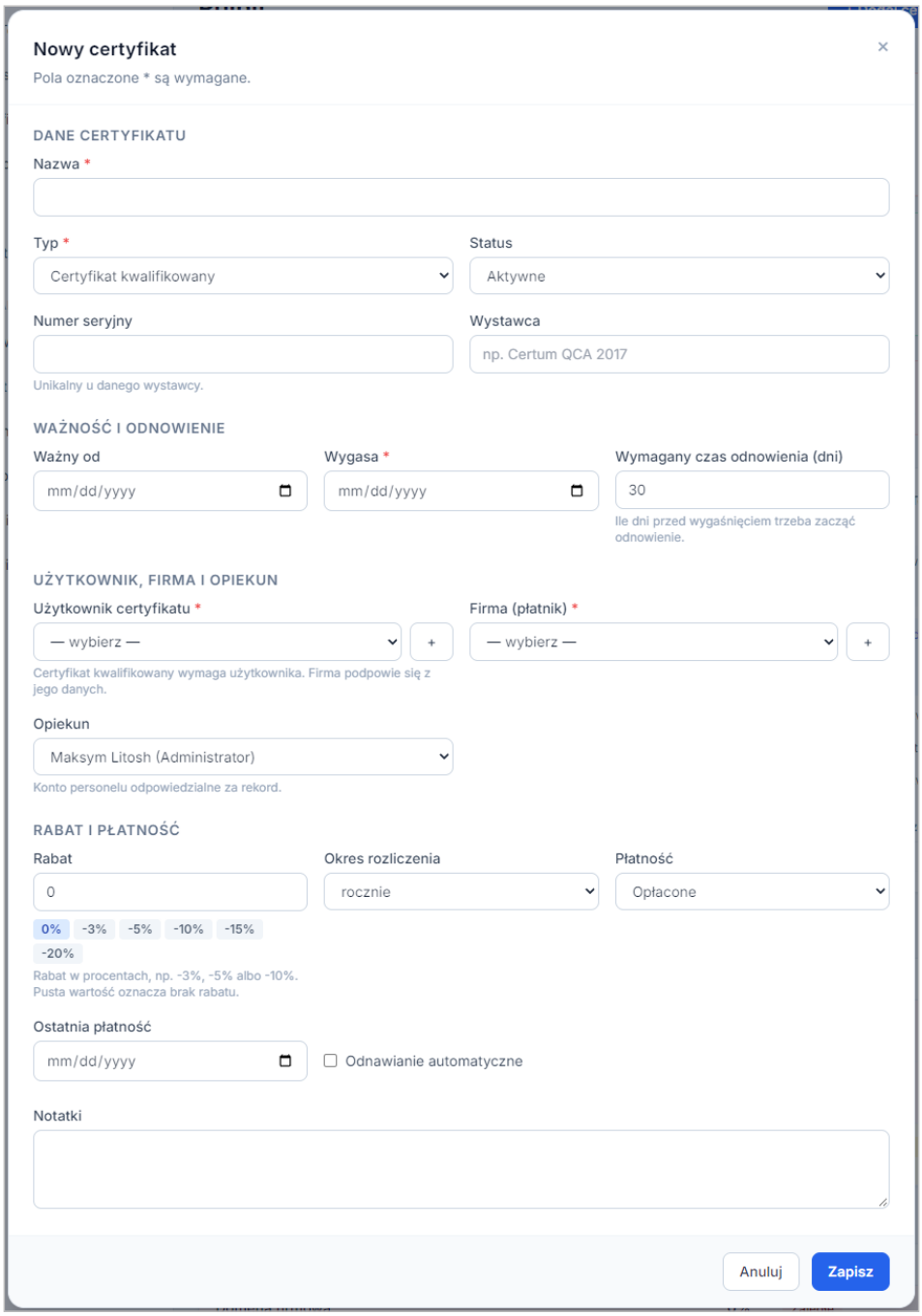


Rysunek 9. Lista certyfikatów z filtrami, rabatem, płatnością i statusem

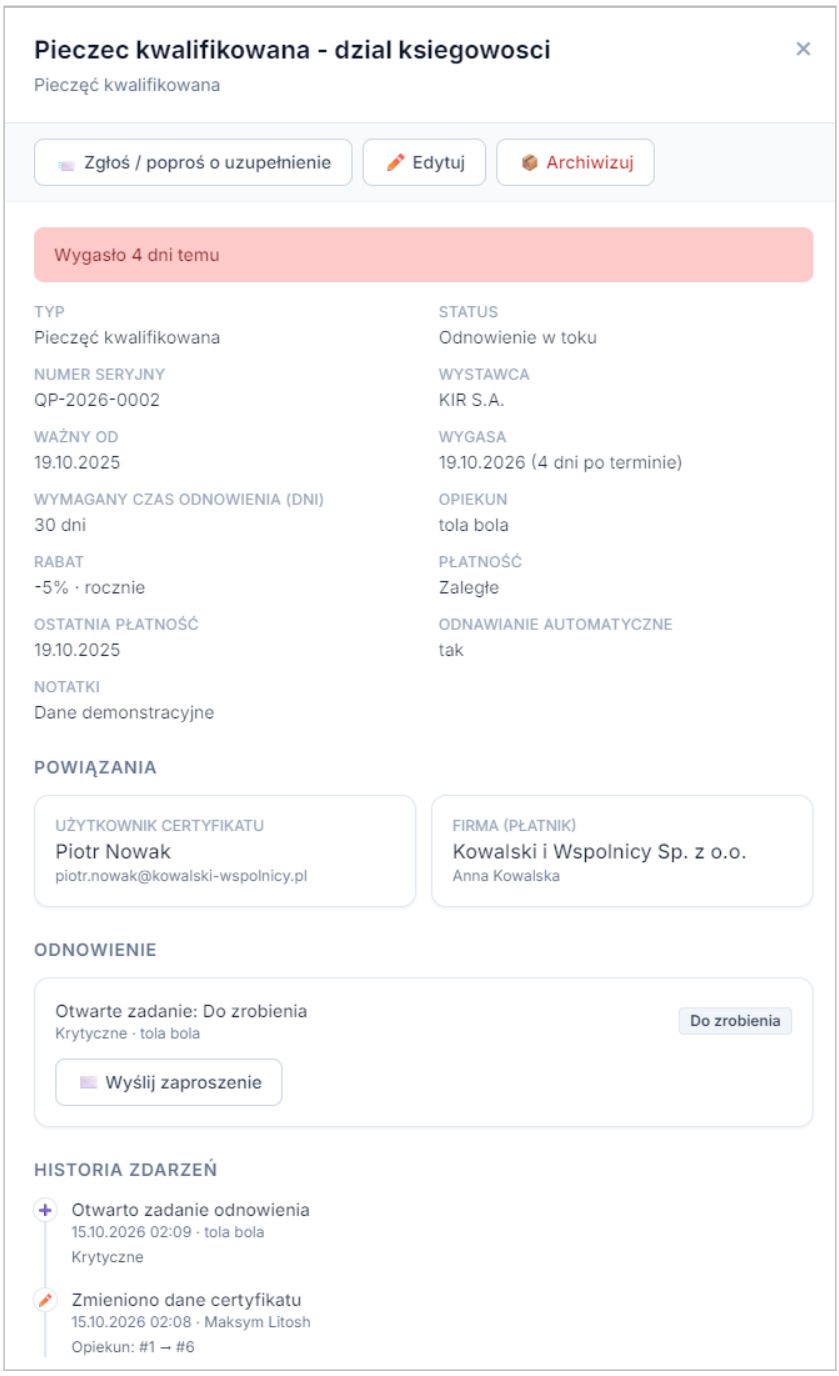


Rysunek 10. Lista użytkowników certyfikatów — każda osoba należy do firmy

Formularze (dodawania i edycji certyfikatu, osoby, firmy) są podzielone na sekcje i pokazują błędy walidacji przy polach, zwracane przez serwer; po wyborze osoby firma podpowiada się sama, a przyciski „+” przy polach pozwalają dodać brakujący rekord bez zamykania formularza. Pole rabatu przyjmuje zapis „-5%”, „-5”, „5” lub „5,5 %” i ma przyciski szybkiego wyboru.

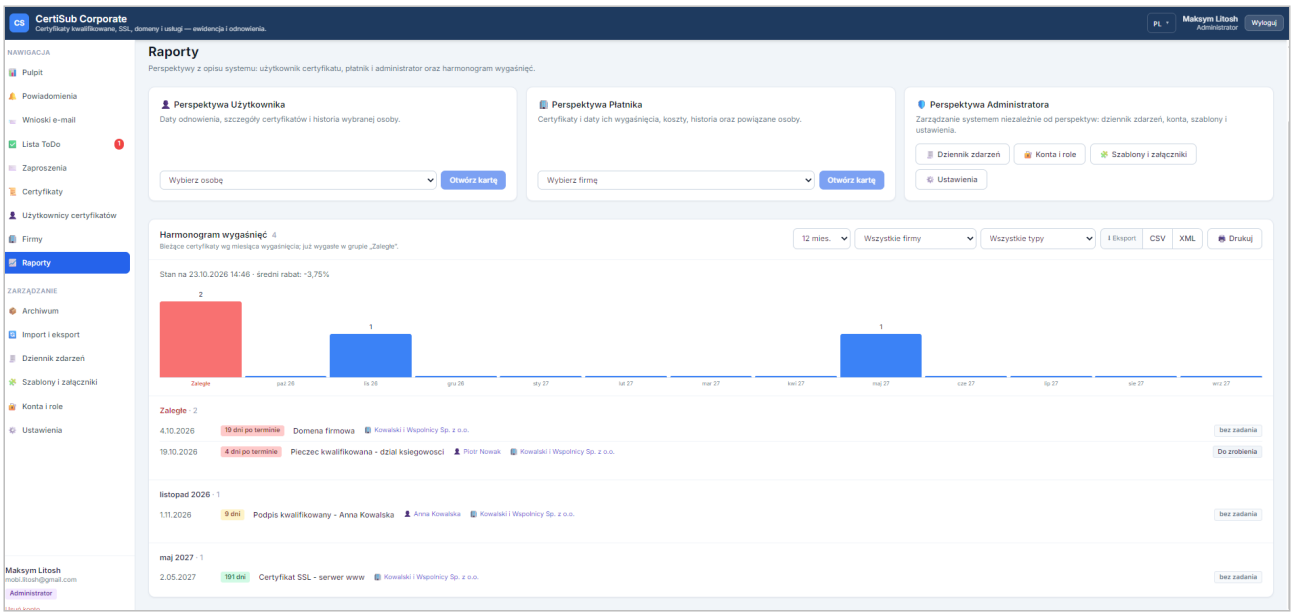


Rysunek 11. Formularz dodawania certyfikatu (sekcje: dane, ważność, powiązania, rabat i płatność)



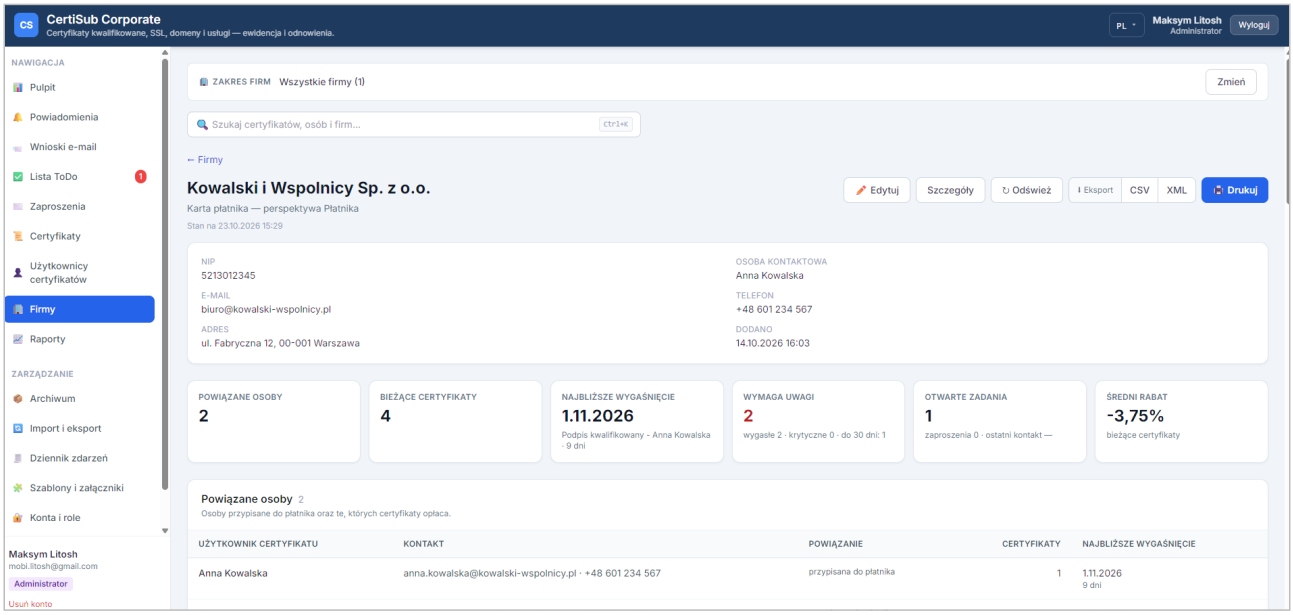
Rysunek 12. Panel szczegółów certyfikatu: powiązania, sekcja „Odnowienie” i historia zdarzeń

Widok „Raporty” (rysunek poniżej) udostępnia perspektywy z opisu pracy: kartę użytkownika certyfikatu, kartę płatnika i skróty administracyjne oraz harmonogram wygaśnięć na 3, 6, 12 lub 24 miesiące z wykresem słupkowym i grupą „Zaległe” dla certyfikatów po terminie. Karty i harmonogram można wydrukować lub zapisać do PDF.

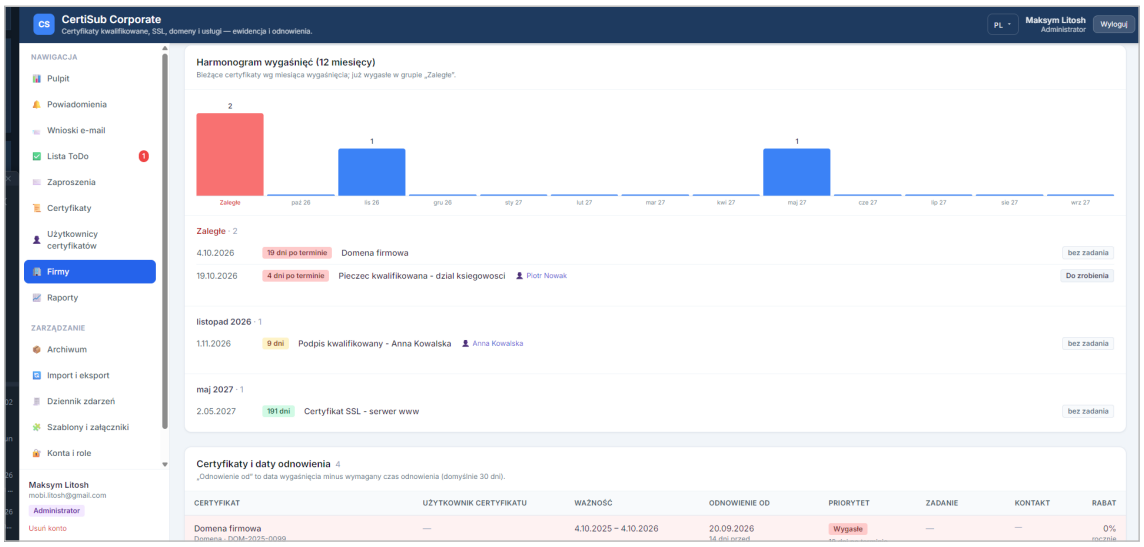


Rysunek 13. Widok „Raporty": perspektywy i harmonogram wygaśnięć

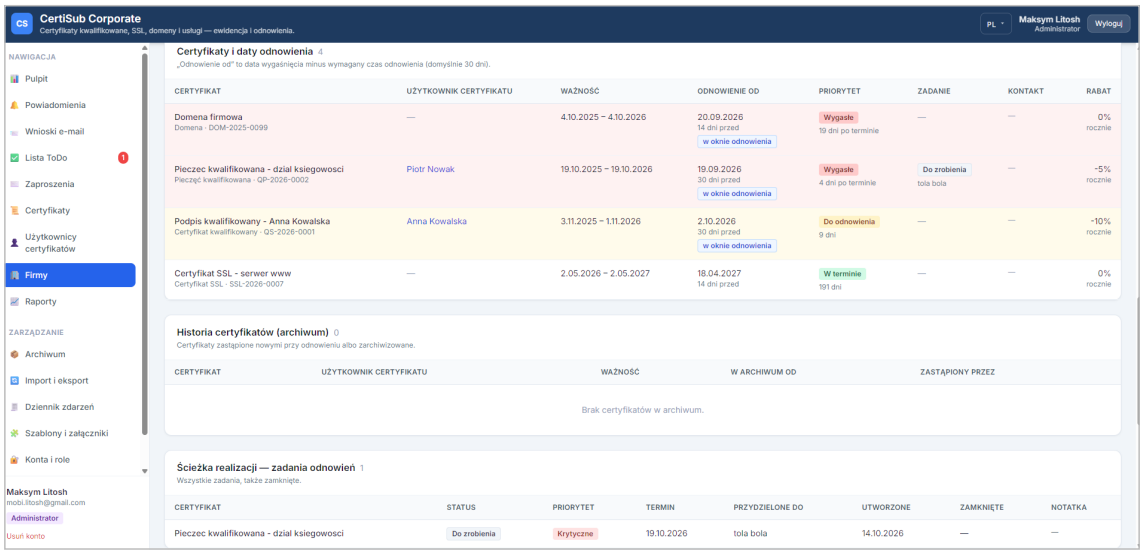
Karta firmy (perspektywa płatnika) zawiera dane kontaktowe, wskaźniki (powiązane osoby, bieżące certyfikaty, najbliższe wygaśnięcie, pozycje wymagające uwagi, otwarte zadania, średni rabat), harmonogram wygaśnięć na 12 miesięcy, tabelę certyfikatów z datą „odnowienie od" i priorytetem, archiwum, ścieżkę realizacji zadań, zaproszenia i oś czasu zdarzeń.



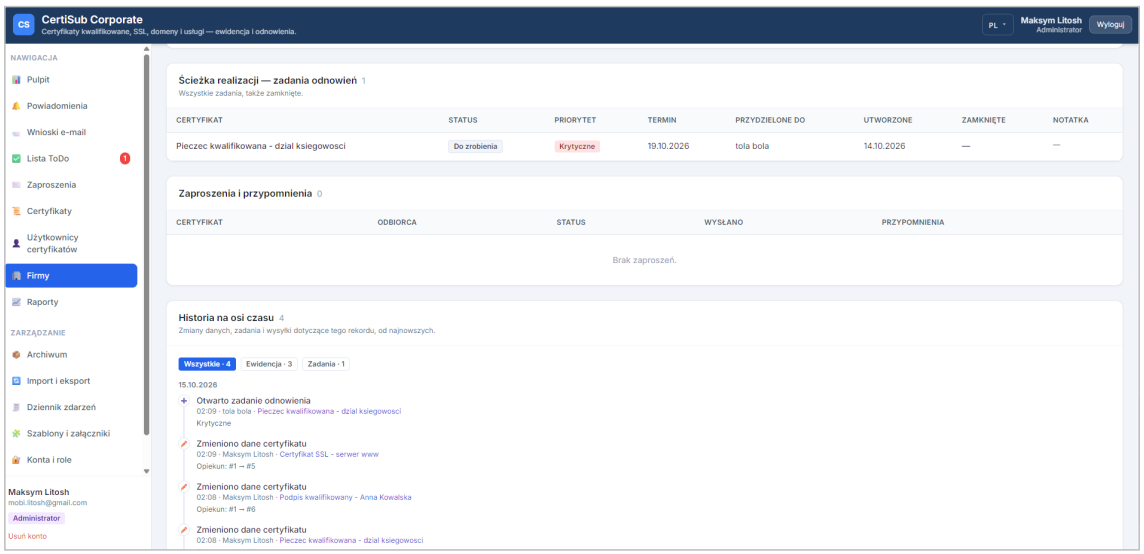
Rysunek 14. Karta firmy: dane kontaktowe, wskaźniki i powiązane osoby



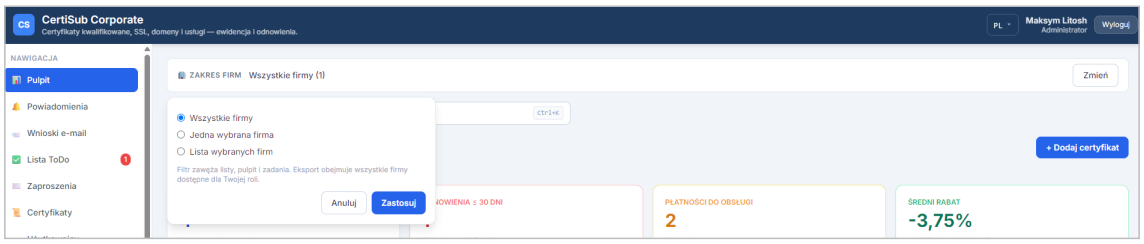
Rysunek 15. Karta firmy: harmonogram wygaśnięć na 12 miesięcy



Rysunek 16. Karta firmy: certyfikaty z datami odnowienia, priorytetem i rabatem



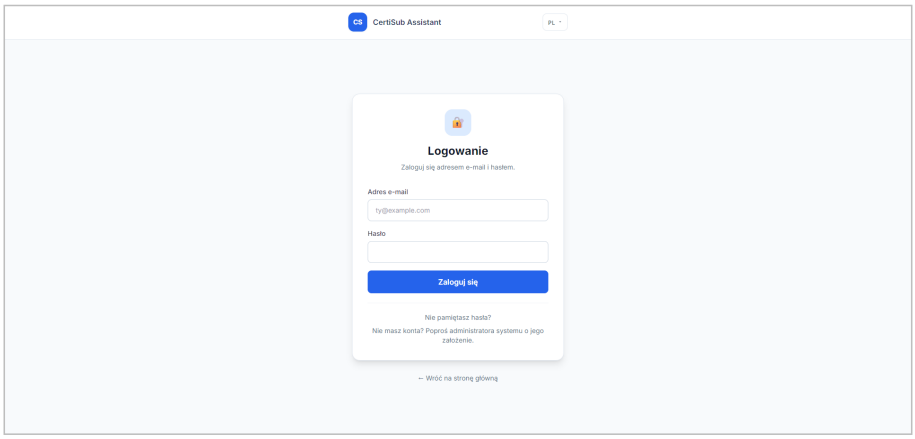
Rysunek 17. Karta firmy: ścieżka realizacji, zaproszenia i historia na osi czasu



Rysunek 18. Pasek „Zakres firm” — okno wyboru: wszystkie firmy, jedna firma albo lista

5.5. System logowania hasłem i weryfikacji adresu e-mail

Pierwsza wersja aplikacji uwierzytelniała personel jednorazowymi kodami wysyłanymi e-mailem (passwordless OTP). W Etapie 9 mechanizm zastąpiono klasycznym logowaniem adresem e-mail i hasłem z jednorazowym potwierdzeniem adresu, co upraszcza pracę kont używanych codziennie i pozwoliło zastosować jednolity limit prób logowania. Tabela login_otps została w bazie jako historia.



Rysunek 19. Ekran logowania

Przepływ uwierzytelniania obejmuje cztery ścieżki:

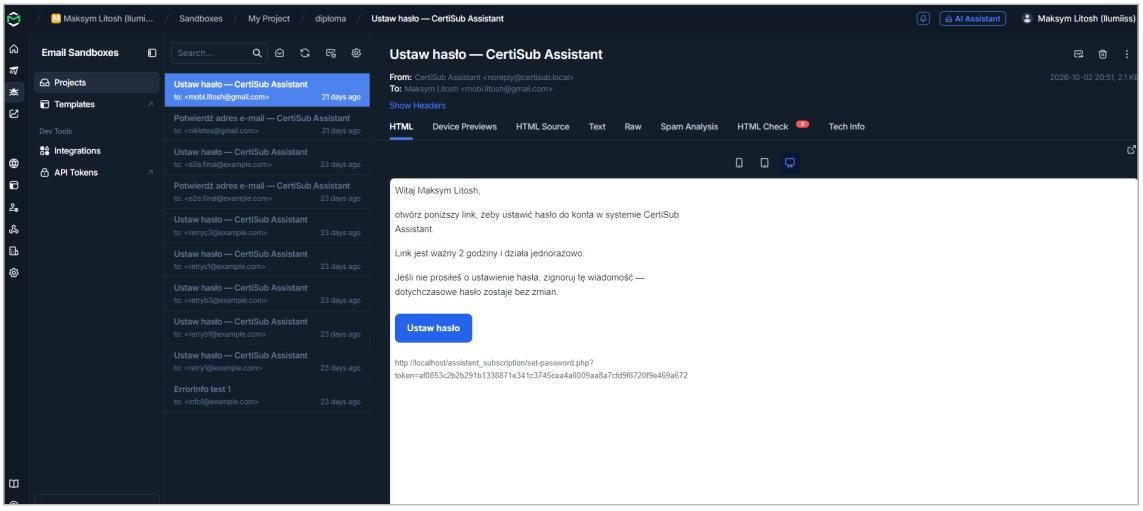
- rejestracja (jeśli włączona w config/auth.php): konto powstaje nieaktywne, a na podany adres trafia jednorazowy link potwierdzający ważny 24 godziny; zapis na zajęty adres wygląda identycznie jak udany, ale niczego nie zmienia i nie wysyła wiadomości,
- konto założone przez administratora: osoba dostaje link do ustawienia hasła ważny 2 godziny — administrator nie zna hasła ani go nie widzi,
- logowanie: ta sama odpowiedź („Nieprawidłowy e-mail lub hasło”) dla błędnego hasła i nieznanego adresu; po pięciu nieudanych próbach na adres lub dwudziestu na adres IP w ciągu 15 minut kolejne próby są blokowane, a udane logowanie zeruje licznik,
- zapomniane hasło: wiadomość z linkiem do ustawienia nowego hasła; zapisanie hasła unieważnia pozostałe linki, a pomyłka w haśle nie „spala” linku.

Polityka haseł (klasa PasswordPolicy) wzorowana jest na wytycznych NIST SP 800-63B [4]: minimum 10 znaków bez wymuszania zestawów znaków i okresowej zmiany, z odrzucaniem haseł łatwych do odgadnięcia. Hasła są zapisywane wyłącznie jako skrót password_hash() i przeliczane automatycznie, gdy zmieni się koszt algorytmu. Tokeny to 32 losowe bajty (random_bytes) przekazywane w linku jako 64 znaki szesnastkowe, a w bazie znajduje się tylko ich skrót SHA-256 z terminem ważności i znacznikiem użycia,

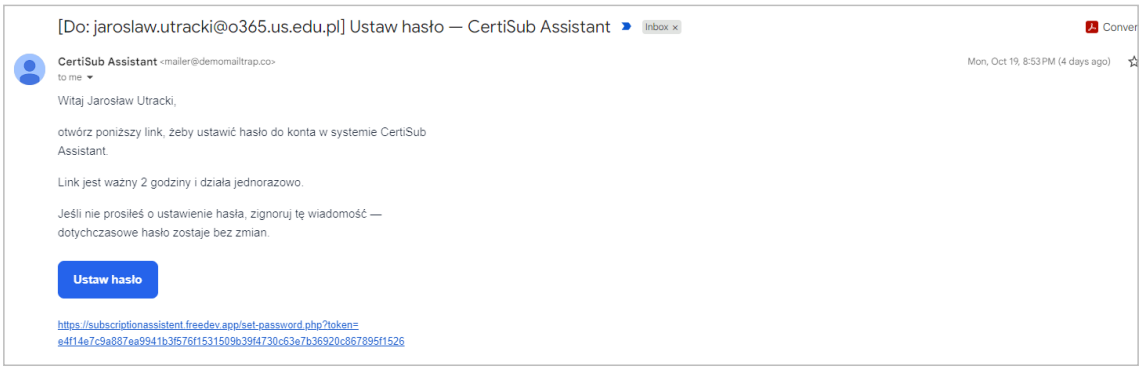
więc kopia bazy nie pozwala użyć cudzego linku. Wysyłki wiadomości weryfikacyjnych są limitowane (3 na adres i 10 na adres IP w 15 minut), a linki budowane są z adresu zapisanego w konfiguracji (klasa AppUrl), nie z nagłówka Host żądania.

Sesja używa ciasteczka HttpOnly z atrybutem SameSite=Lax (oraz Secure przy HTTPS), po zalogowaniu identyfikator sesji jest regenerowany, a wyłączenie konta kończy sesję przy następnym żądaniu. Wszystkie zapisy wymagają tokenu CSRF porównywanego funkcją hash_equals. Po zalogowaniu przekierowanie jest dozwolone wyłącznie na stronę z listy dozwolonych celów. Wiadomości wychodzą przez SMTP z autoryzacją OAuth2 (XOAUTH2) — własna implementacja interfejsu OAuthTokenProvider biblioteki PHPMailer wymienia refresh token na token dostępu bez hasła do skrzynki w konfiguracji [21].

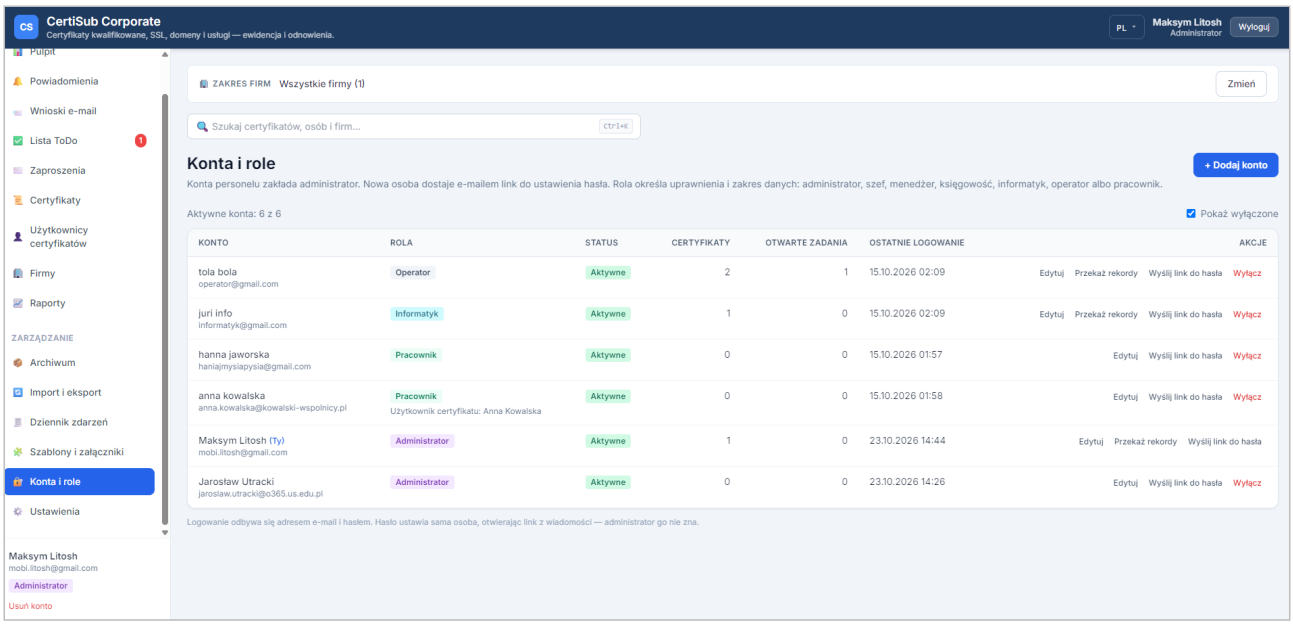
Wiadomość z linkiem do ustawienia hasła w trybie sandbox trafia do skrzynki Mailtrap (pierwszy rysunek poniżej), a w wersji demonstracyjnej jest dostarczana do rzeczywistej skrzynki odbiorcy (drugi rysunek). Link jest jednorazowy i ważny 2 godziny, a w bazie zapisany jest tylko skrót tokenu.



Rysunek 20. Wiadomość „Ustaw hasło” w skrzynce Mailtrap (sandbox)



Rysunek 21. Ta sama wiadomość dostarczona do rzeczywistej skrzynki odbiorcy (wersja demonstracyjna)



Rysunek 22. Widok „Konta i role” dla administratora

Widok „Konta i role” (rysunek powyżej) pozwala administratorowi zakładać konta z rolą, edytować je, przekazywać rekordy odchodzącego pracownika innemu kontu, wysyłać link do ustawienia hasła oraz wyłączać konta. System nie pozwala wyłączyć własnego konta ani ostatniego aktywnego administratora.

6. Moduł zadań, zaproszeń i przypomnień e-mail

6.1. Założenia procesu odnowień i zadań ToDo

Proces odnowień jest sercem systemu i realizuje wymagania F11, F12, F18 z opisu pracy. Skaner (klasa RenewalScanner) przegląda bieżące certyfikaty i dla każdego, który wszedł w margines odnowienia, zakłada zadanie ToDo przypisane opiekunowi. Margines to wymagany czas odnowienia zapisany przy certyfikacie (renewal_lead_days), a gdy go brak — domyślny próg z ustawień (30 dni). Priorytet zadania zależy od liczby dni do wygaśnięcia: „wygasłe” po dacie, „krytyczne” przy progu krytycznym (domyślnie 7 dni), „do odnowienia” w marginesie.

Listing 3. Warunek wyboru certyfikatów przez skaner (fragment zapytania SQL)

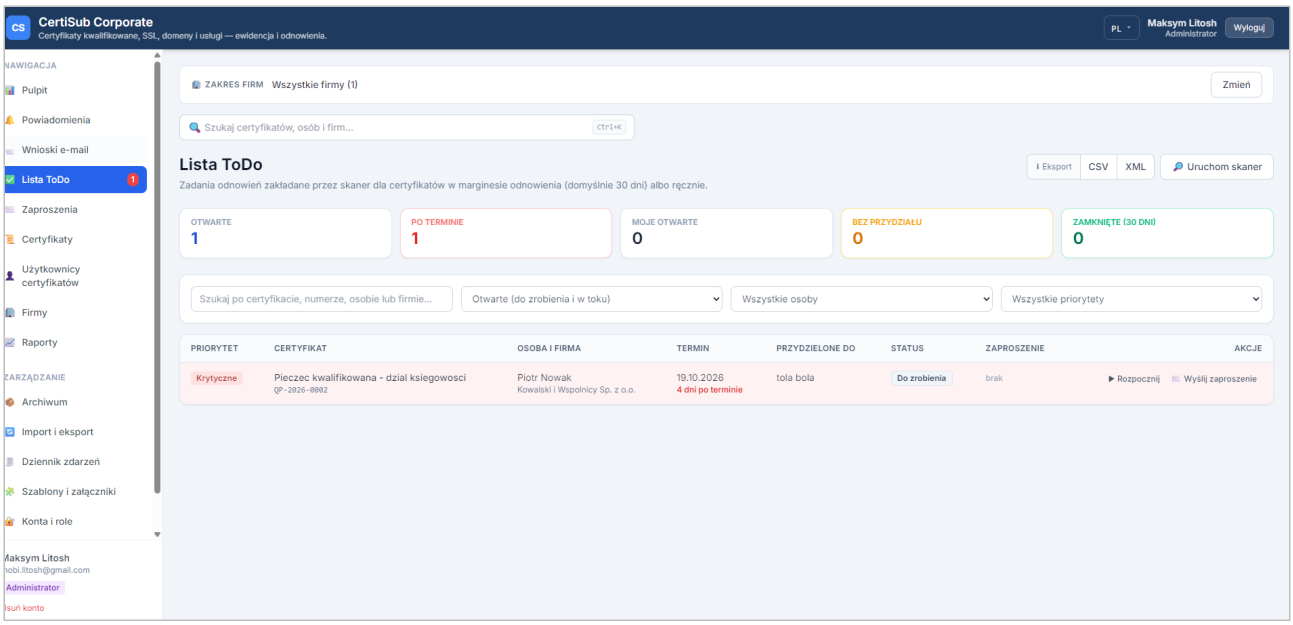
```
WHERE c.archived_at IS NULL
  AND c.expiry_date <= DATE_ADD(:today,
        INTERVAL COALESCE(c.renewal_lead_days, :default_lead) DAY)
  AND (t.id IS NOT NULL OR NOT EXISTS (
       SELECT 1 FROM renewal_tasks handled
       WHERE handled.certificate_id = c.id
         AND handled.status IN ('done', 'abandoned')
         AND handled.due_date = c.expiry_date))
```

Skaner jest idempotentny. Certyfikat ma co najwyżej jedno zadanie otwarte (gwarantuje to unikalny indeks na kolumnie wyliczanej open_marker, opisany w podrozdziale 4.3), a zamknięte zadanie o terminie równym dacie wygaśnięcia oznacza „cykl już obsłużony”, więc kolejne uruchomienie tego samego dnia niczego nie powiela. Zadanie certyfikatu, którego opiekun ma wyłączone konto, pozostaje bez przydziału do czasu decyzji menedżera.

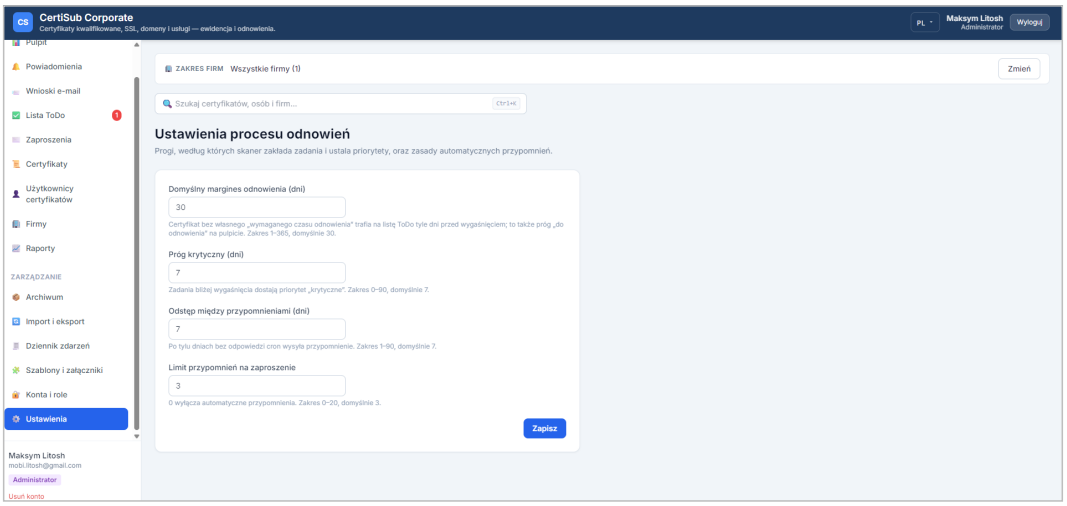
Tabela 4. Statusy zadania odnowienia i dozwolone przejścia (klasa TaskService)

Status	Znaczenie	Dozwolone przejścia
todo	do zrobienia — zadanie założone przez skaner lub ręcznie	w toku, zrobione, porzucone
in_progress	w toku — rozpoczęto prace, np. wysłano zaproszenie	do zrobienia, zrobione, porzucone
done	zrobione — certyfikat odnowiono	do zrobienia (tylko menedżer)
abandoned	porzucone — wymagane uzasadnienie	do zrobienia (tylko menedżer)

Ekran „Lista ToDo” (rysunek poniżej) pokazuje liczniki (otwarte, po terminie, moje otwarte, bez przydziału, zamknięte w ostatnich 30 dniach), filtry statusu, osoby i priorytetu oraz tabelę zadań z akcjami „Rozpocznij” i „Wyślij zaproszenie”. Zakończenie zadania akcją „Odnów certyfikat” tworzy nowy certyfikat z tymi samymi powiązaniami (osoba, firma, opiekun) i nową datą wygaśnięcia, archiwizuje poprzedni i łączy oba w łańcuch odnowień widoczny w historii. Administrator może zmienić progi (margines, próg krytyczny, odstęp i limit przypomnień) w widoku „Ustawienia”; każda zmiana trafia do historii zdarzeń.



Rysunek 23. Lista ToDo: zadanie o priorytecie „Krytyczne” po terminie

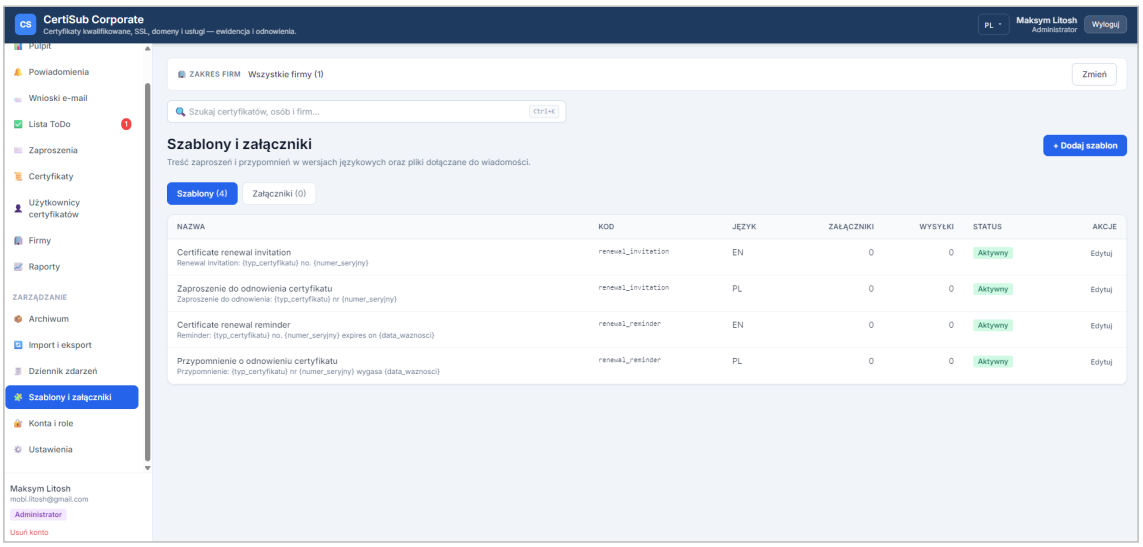


Rysunek 24. Widok „Ustawienia” — progi procesu odnowień

6.2. Szablony wiadomości, załączniki i renderowanie treści

Zaproszenia i przypomnienia powstają z szablonów edytowanych przez administratora. Szablon ma kod (renewal_invitation lub renewal_reminder), język, temat oraz treść HTML i tekstową z polami w nawiasach klamrowych: {imie}, {nazwisko}, {typ_certyfikatu}, {numer_seryjny}, {data_waznosci}, {dni_do_wygasniecia} i {platnik}. System dostarcza cztery szablony domyślne (zaproszenie i przypomnienie, po polsku i po angielsku). Klasa TemplateRenderer podstawia pola, przy czym w treści HTML wartości są escapowane, a w temacie usuwane są znaki sterujące, co zapobiega wstrzyknięciu kodu i nagłówków przez dane certyfikatu.

Załączniki przechowuje biblioteka plików (PDF, DOC/DOCX, ODT, XLSX, TXT, CSV, PNG, JPG, ZIP do 10 MB). Aplikacja sprawdza rozszerzenie i rzeczywistą zawartość pliku, zapisuje go pod losową nazwą w katalogu storage/attachments niedostępnym z przeglądarki i zapamiętuje skrót SHA-256; plik użyty w szablonie lub wysłany w zaproszeniu nie może zostać usunięty.



Rysunek 25. Ekran „Szablony i załączniki” — cztery szablony domyślne (PL i EN)

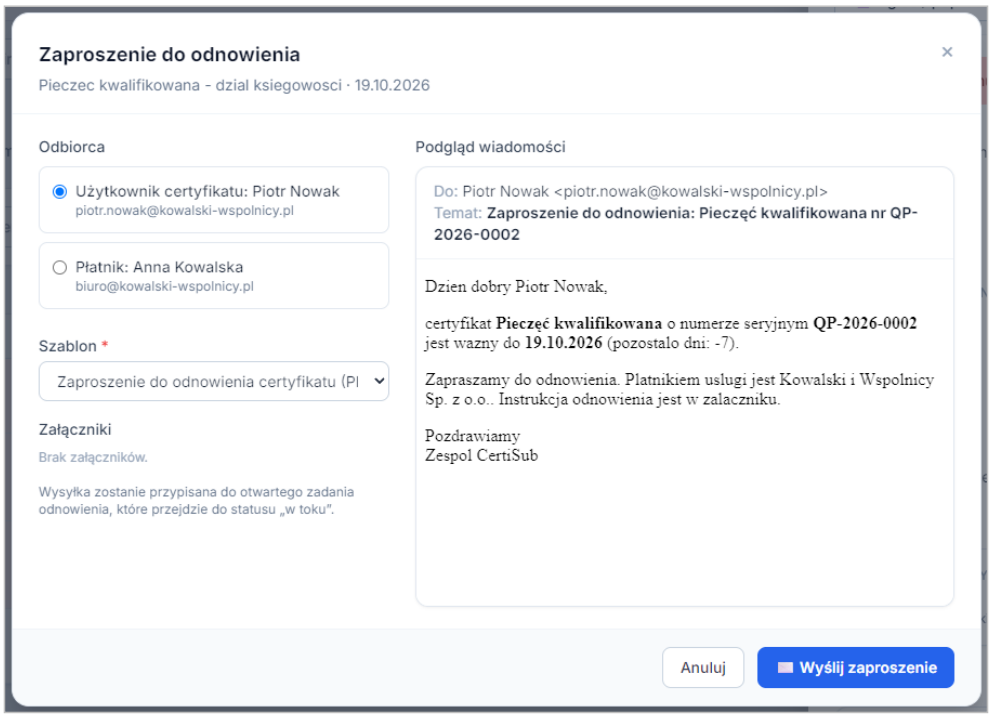
6.3. Mechanizm cron, wysyłka zaproszeń i przypomnień

Okno wysyłki zaproszenia pozwala wybrać odbiorcę (użytkownik certyfikatu albo firma — odbiorca bez adresu e-mail jest wyszarzony), szablon, załączniki (domyślnie przypisane do szablonu) i wyświetla podgląd treści wypełnionej danymi certyfikatu. Wysyłka zapisuje wiersz w rejestrze invitations z treścią z chwili wysyłki, ustawia zadanie w stan „w toku” (a gdy zadania nie było — zakłada je) i planuje termin pierwszego przypomnienia. Wiadomość wysyła klasa Mailer (PHPMailer) za pośrednictwem interfejsu InvitationMailer, który w testach zastępuje rejestrujący zamiennik — żaden test nie wysyła prawdziwej poczty. Nieudana wysyłka zostaje w rejestrze ze statusem „failed” i treścią błędu SMTP, a przycisk „Ponów” pozwala ją powtórzyć. Darmowy plan usługi Mailtrap przyjmuje jedną wiadomość na sekundę, dlatego Mailer po odrzuceniu z tego powodu czeka dwie sekundy i ponawia wysyłkę raz, a każdy błąd jest zapisywany z przyczyną w logs/mail-errors.log.

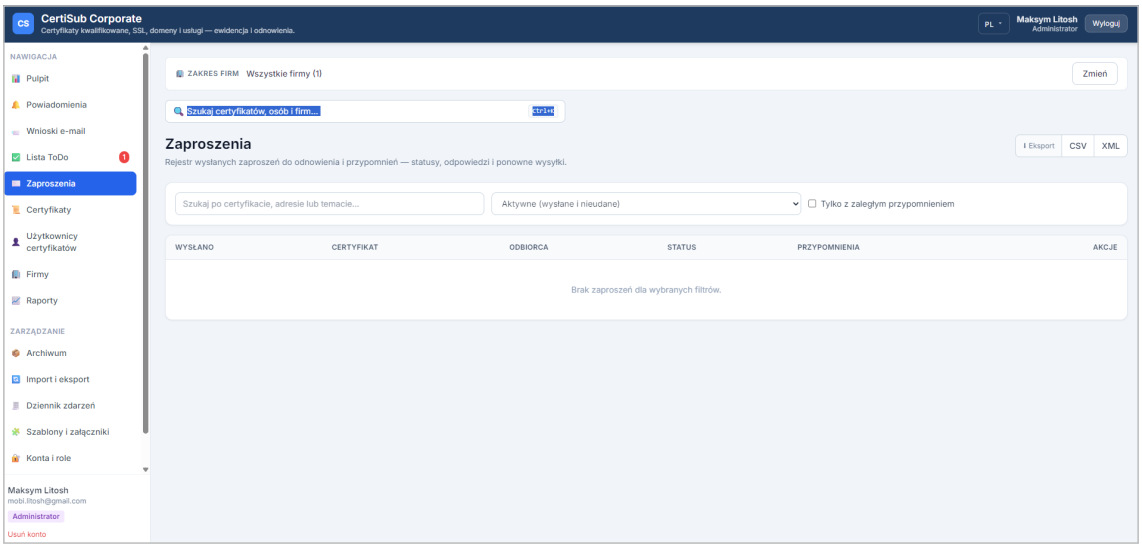
Tabela 5. Statusy zaproszenia (tabela invitations)

Status	Znaczenie
queued	zaplanowane, jeszcze niewysłane
sent	wysłane; cron wysyła kolejne przypomnienia w ustalonym odstępie do limitu
failed	wysyłka nieudana (treść błędu w rejestrze), możliwe ponowienie
responded	odbiorca odpowiedział — automatyczne przypomnienia wstrzymane
closed	zamknięte ręcznie albo przez zakończenie lub porzucenie zadania

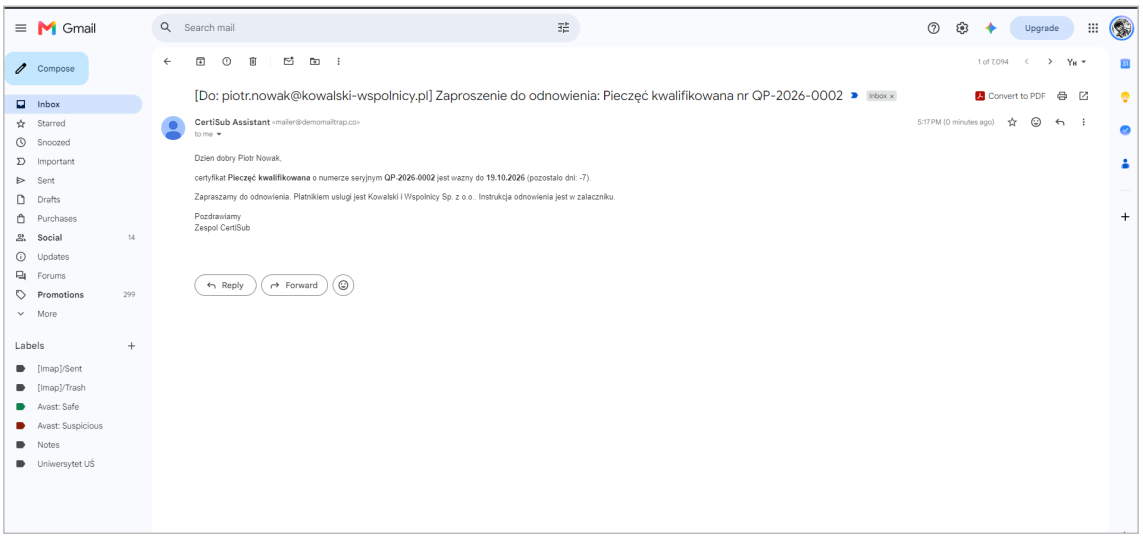
Zadania okresowe uruchamia jeden skrypt, cron/renewals.php (raz na dobę, np. o 7:00): najpierw skaner odnowień (zakłada zadania i podnosi priorytety), potem wysyłka zaległych przypomnień według ustawień (domyślnie co 7 dni, nie więcej niż 3 na zaproszenie; wartość 0 wyłącza przypomnienia automatyczne). Wynik trafia do logs/renewals.log i do historii zdarzeń, a skrypt uruchomiony z przeglądarki zwraca 403. Skaner może też uruchomić ręcznie menedżer lub administrator przyciskiem „Uruchom skaner”.



Rysunek 26. Okno „Wyślij zaproszenie": wybór odbiorcy, szablon i podgląd treści z podstawionymi danymi certyfikatu



Rysunek 27. Rejestr „Zaproszenia" z filtrami statusu i zaległych przypomnień

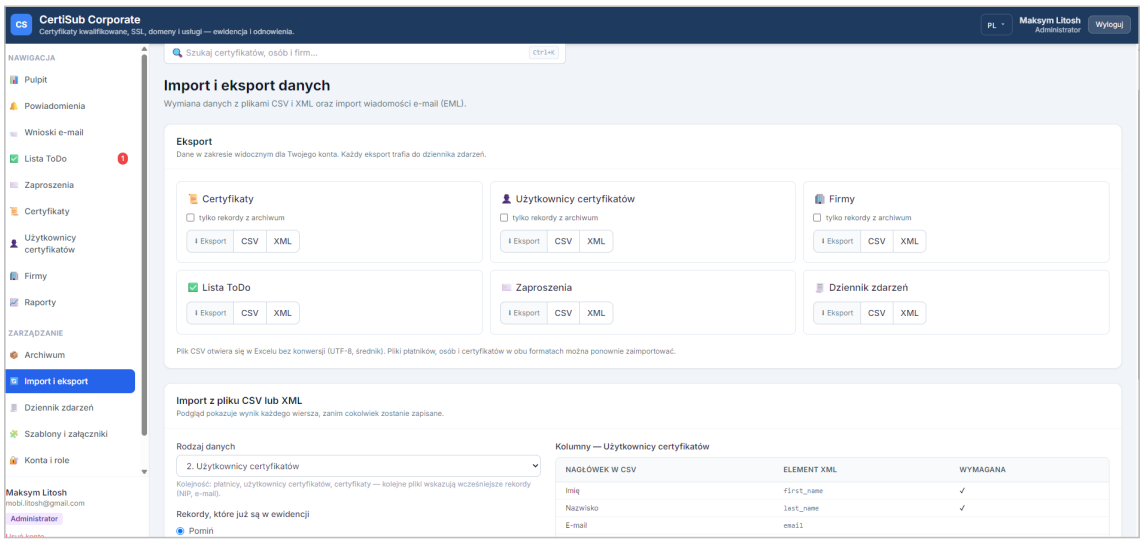


Rysunek 28. Zaproszenie dostarczone do skrzynki odbiorcy z wersji demonstracyjnej na hostingu

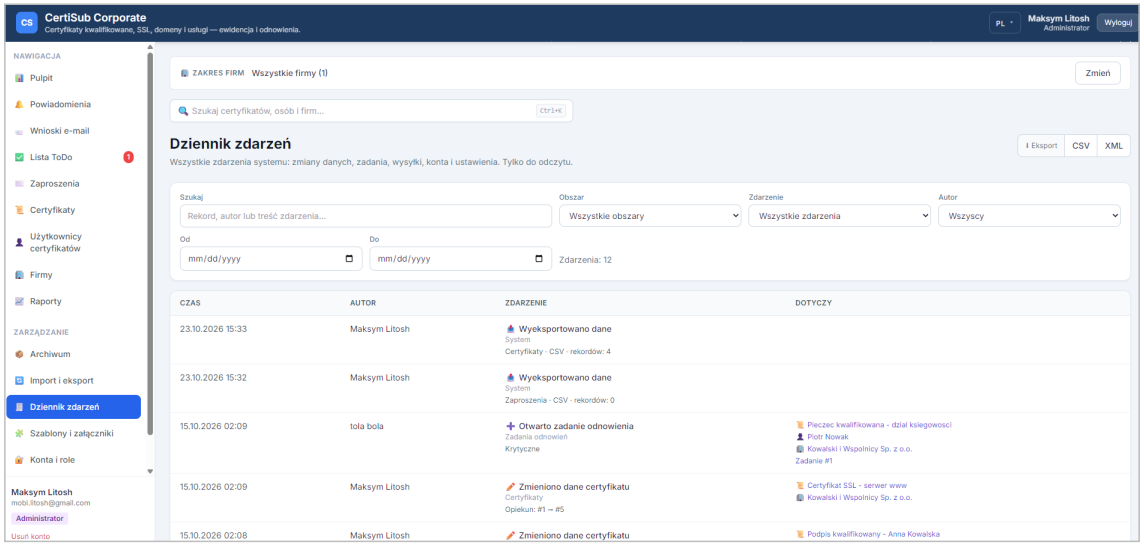
6.4. Integracja modułu z aplikacją webową i rejestr zdarzeń

Moduł jest zintegrowany z resztą aplikacji przez wspólne usługi. Każde zdarzenie procesu — założenie zadania, zmiana statusu lub priorytetu, przydział, wysyłka, przypomnienie, odnowienie — zapisuje wiersz w tabeli events z identyfikatorami kontekstu, dlatego ta sama historia pojawia się w szczegółach certyfikatu, na karcie użytkownika certyfikatu, na karcie firmy i w dzienniku administratora. Oś czasu pokazuje zdarzenia pogrupowane według dni z filtrem rodzaju (ewidencja, zadania, zaproszenia), a TimelineService ukrywa zdarzenia certyfikatów spoza zakresu konta. Przydzielenie zadania wysyła komunikat systemowy do wskazanej osoby.

Wymianę danych obsługuje moduł App\Exchange. Eksport CSV (UTF-8 z BOM, separator średnik, apostrof przed treścią zaczynającą się od =, +, - lub @, aby arkusz nie wykonał jej jako formuły) i XML obejmuje wyłącznie rekordy w zakresie danych konta, a samo pobranie zapisuje zdarzenie w dzienniku. Import (tylko administrator) wykonuje dokładnie ten sam zapis przez usługi co ręczne dodawanie, wewnątrz transakcji z punktem zapisu na każdy wiersz, a w trybie podglądu wycofuje całość — podgląd pokazuje więc rzeczywisty wynik, a nie prognozę, i nie może się rozjechać z importem. Import wiadomości EML (własny parser MIME, bez rozszerzenia mailparse) dopasowuje nadawcę do osoby lub firmy, certyfikaty po numerach seryjnych i oznacza odpowiedzi na zaproszenia.



Rysunek 29. Widok „Import i eksport” — eksport zbiorów i kreator importu



Rysunek 30. Dziennik zdarzeń administratora z filtrami

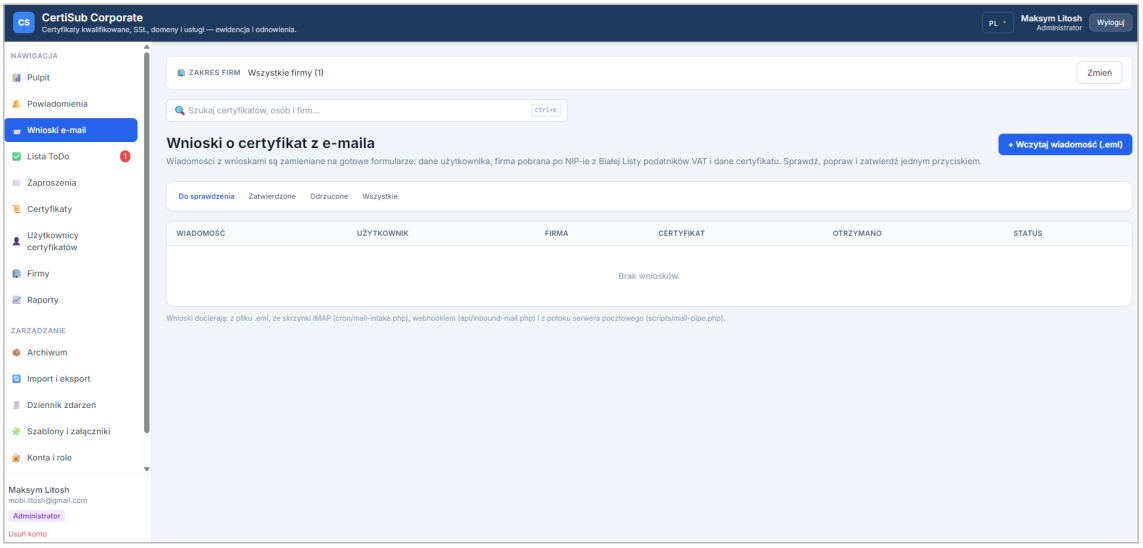
6.5. Wnioski o certyfikat przesłane e-mailem

W Etapie 10 dodano moduł skracający rejestrację certyfikatów zamawianych pocztą. Wiadomość z wnioskiem trafia do aplikacji jedną z czterech dróg, które przechodzą przez wspólną metodę RegistrationIntakeService::ingest(): przez przesłanie pliku .eml w panelu, ze skrzynki IMAP (cron/mail-intake.php, własny klient IMAP z autoryzacją hasłem lub OAuth2) [19], przez webhook api/inbound-mail.php chroniony tokenem w nagłówku X-Intake-Token oraz potokiem serwera pocztowego (scripts/mail-pipe.php).

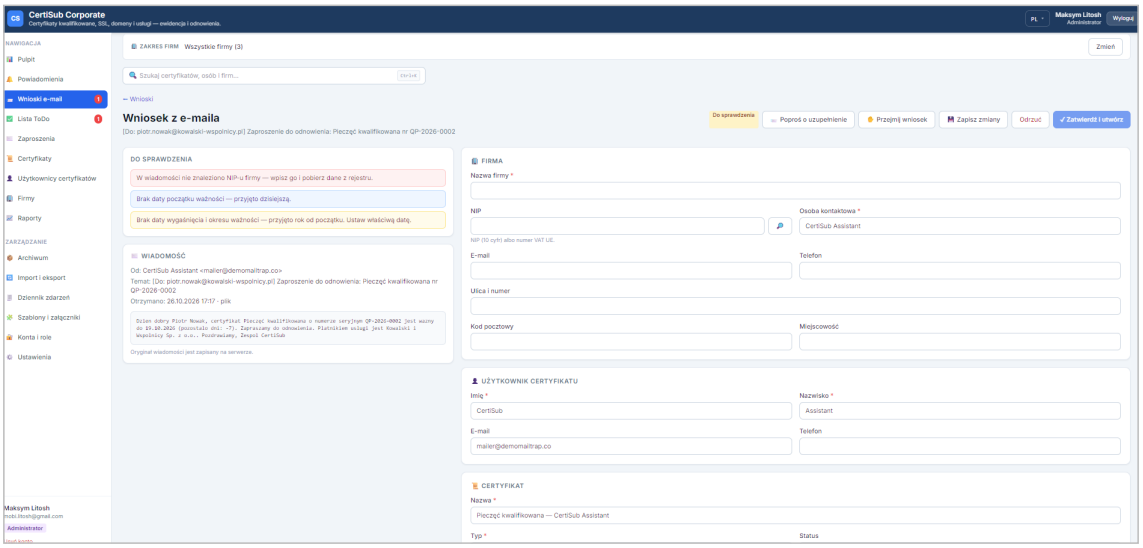
Klasa RegistrationExtractor jest czystą funkcją wiadomości: rozpoznaje etykiety „Etykieta: wartość” po polsku i angielsku, nagłówki sekcji („Dane firmy”, „Dane wnioskodawcy”), wiersze tabel HTML, pary rozdzielone tabulatorem lub kreską pionową oraz NIP wpisany w zdaniu (zawsze sprawdzany sumą kontrolną). Ponieważ skrzynka przyjmuje pocztę od obcych, ekstraktor ogranicza ilość czytanych danych (tekst 200 kB, HTML 300 kB, 4000 linii, 500 wierszy tabeli), a analiza tabel ma złożoność liniową; testy sprawdzają, że wrogi dokument przetwarza się szybko, a nadmiar jest obcinany. Wynik jest propozycją, nie danymi zaufanymi: każde pole zapisuje źródło (etykieta, nagłówek, treść, wartość

domyślna), a wątpliwości trafiają na listę ostrzeżeń.

Dane firmy uzupełnia klient Białej Listy podatników VAT Ministerstwa Finansów [18] — publiczne API bez klucza, odpytywane po NIP-ie; firma, której nie ma w rejestrze (subject = null), błąd rejestru albo limit zapytań nie blokują wniosku, tylko dodają ostrzeżenie. Wynik (tabela registration_drafts) trafia do kolejki operatora, który widzi treść wiadomości obok wypełnionego formularza firmy, osoby i certyfikatu oraz listę uwag. Zatwierdzenie jednym przyciskiem zakłada w jednej transakcji firmę (jeśli jej nie było), osobę w tej firmie i certyfikat z operatorem jako opiekunem; błąd w dowolnym kroku wycofuje całość i podświetla pole w odpowiedniej sekcji. Do rejestru ewidencji nic nie trafia przed decyzją człowieka, a ta sama wiadomość (Message-ID lub skrót treści) nie powstaje drugi raz.



Rysunek 31. Kolejka „Wnioski e-mail” (zakładki: do sprawdzenia, zatwierdzone, odrzucone)



Rysunek 32. Ekran sprawdzenia wniosku: treść wiadomości i lista uwag obok formularza — w tej wiadomości nie było NIP-u, więc danych firmy nie dało się pobrać z rejestru

6.6. Dane firm z otwartych rejestrów publicznych

Dane firmy po numerze podatkowym pobiera osobna warstwa, wspólna dla całej aplikacji. Każde źródło implementuje interfejs CompanyRegistry z dwiema metodami: supports()

rozstrzyga, czy potrafi odpowiedzieć na dany numer, a lookupNip() zwraca rekord firmy albo null, gdy numeru nie ma w rejestrze. CompositeCompanyRegistry pyta wyłącznie te źródła, które obsługują dany numer, więc polski NIP trafia do Białej Listy podatników VAT [18], a numer VAT z kodem kraju do unijnego systemu VIES [24]; awaria jednego źródła nie przerywa łańcucha, a gdy nie odpowiedziało żadne, operator widzi komunikat o awarii, a nie informację o braku firmy. Oba interfejsy są publiczne i nie wymagają klucza, co było warunkiem wdrożenia na bezpłatnym hostingu.

Zakres danych z każdego źródła jest różny. Biała Lista zwraca nazwę, status VAT, REGON, numer KRS i adres, VIES — nazwę i adres, przy czym część państw członkowskich (m.in. Niemcy, Hiszpania i Włochy) potwierdza wyłącznie ważność numeru i zamiast danych podatnika zwraca znacznik „---”; aplikacja zamienia go na puste pole, a operator dostaje potwierdzenie numeru wraz z ostrzeżeniem, że dane trzeba wpisać ręcznie. Jeżeli Biała Lista poda numer KRS, odpytywany jest dodatkowo rejestr sądowy [25], który uzupełnia formę prawną i adres rozbity na ulicę, kod pocztowy i miejscowość. Dział drugi odpisu, zawierający dane osób reprezentujących spółkę, nie jest czytany: ewidencja certyfikatów go nie potrzebuje, a ograniczenie zakresu pobieranych danych osobowych wynika z zasady minimalizacji [2].

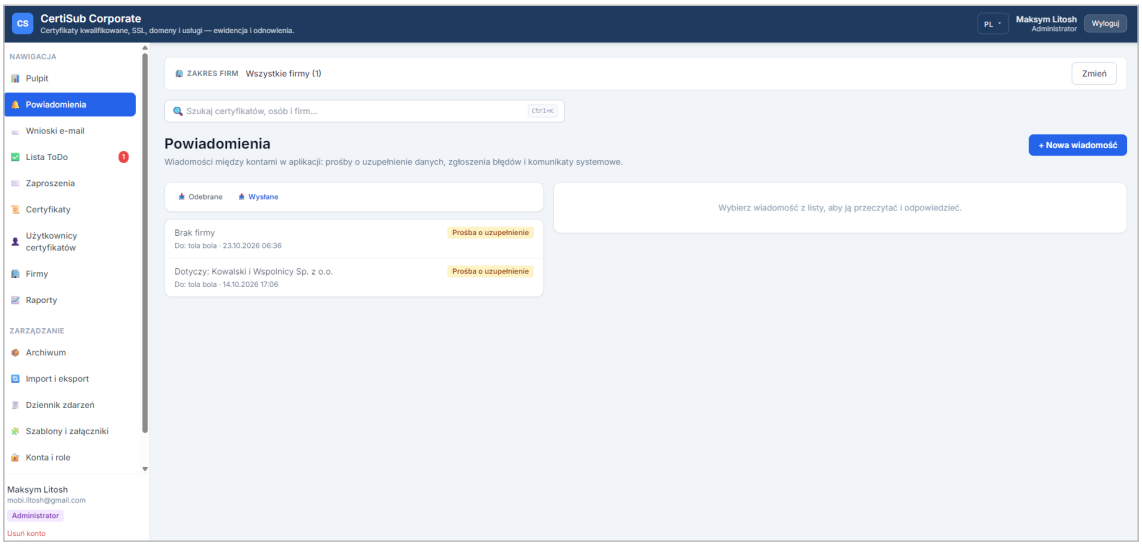
Ponieważ rejestry są usługami zewnętrznymi o ograniczonej przepustowości, zapytania przechodzą przez dwa dekoratory. CachedCompanyRegistry trzyma odpowiedzi w pamięci podręcznej przez siedem dni i zapamiętuje także odpowiedź „nie ma takiego numeru”, natomiast awarii nie zapamiętuje — chwilowa niedostępność rejestru ma prawo minąć przed kolejną próbą. LookupThrottle ogranicza liczbę zapytań do dwudziestu na konto na godzinę, co chroni zarówno przed limitem po stronie rejestru, jak i przed pętlą w interfejsie; brak katalogu do zapisu licznika nie blokuje funkcji, bo ograniczenie ma charakter ochronny, a nie kontroli dostępu.

Z tej samej warstwy korzystają trzy miejsca w aplikacji. Pierwsze to opisana wyżej kolejka wniosków z poczty. Drugie to formularz firmy, w którym przycisk „Pobierz z rejestru” przy polu NIP wypełnia nazwę i adres, a pod polami pokazuje dane rejestrowe (status VAT, REGON, KRS, datę rejestracji) oraz ostrzeżenia: firmy nie ma w rejestrze, status VAT jest nieczynny, adres jest niepełny albo firma jest już w ewidencji. Pola wypełnione wcześniej przez operatora nadpisywane są dopiero po potwierdzeniu. Trzecie to import plików CSV i XML: gdy wiersz ma numer podatkowy, a komórka z nazwą lub adresem jest pusta, brakująca wartość pochodzi z rejestru — danych z pliku aplikacja nigdy nie nadpisuje, bo plik jest deklaracją operatora. Uzupełnienie działa już na etapie podglądu importu (inaczej podgląd pokazywałby błędy walidacji, których przy zapisie nie będzie), a jeden plik może wywołać najwyżej sto zapytań. We wszystkich trzech ścieżkach obowiązuje ta sama zasada: dane z rejestru są propozycją i nie trafiają do ewidencji bez decyzji człowieka.

6.7. Powiadomienia wewnętrzne między kontami

Powiadomienia służą głównie do komunikacji administratora z operatorami. Jeden wiersz w tabeli notifications to jedna wiadomość dla jednego odbiorcy (wiadomość do wielu osób to kilka wierszy o wspólnym kluczu batch_key); odpowiedzi tworzą wątek (thread_id,

parent_id). Rodzaje: wiadomość, prośba o uzupełnienie danych, zgłoszenie błędu oraz komunikat systemowy bez nadawcy (np. „przydzielono Ci zadanie odnowienia”, „nowy wniosek z e-maila”). Przycisk „Zgłoś / poproś o uzupełnienie” w szczegółach certyfikatu, osoby lub firmy powiązuje wiadomość z rekordem, który odbiorca otwiera jednym kliknięciem; prośbę można oznaczyć jako załatwioną, co powiadamia nadawcę.



Rysunek 33. Widok „Powiadomienia” — prośby o uzupełnienie danych

7. Testowanie i ocena systemu

7.1. Scenariusze testowe

Strategia testowania opiera się na trzech poziomach. Testy jednostkowe (katalog tests/Unit) sprawdzają logikę bez bazy danych albo na bazie SQLite w pamięci. Testy integracyjne (tests/Integration) uruchamiają prawdziwe usługi na jednorazowej bazie MySQL assistent_subscriptions_test, tworzonej od zera ze schematu i migracji przy każdym przebiegu, więc nigdy nie dotykają danych użytkownika; są pomijane, dopóki nie ustawi się zmiennej RUN_INTEGRATION_TESTS. Scenariusz end-to-end przechodzi przez warstwę API, czyli przez to samo jądro, które obsługuje żądania HTTP (metoda, CSRF, sesja, kody odpowiedzi). Poczta w testach jest zastępowana rejestrującym zamiennikiem, a rejestr VAT i serwer IMAP — atrapami, dzięki czemu testy są powtarzalne i nie wymagają sieci.

Scenariusze zostały dobrane tak, aby pokryć wymagania F1–F19 i N1–N6 oraz najbardziej ryzykowne miejsca: uprawnienia, spójność danych, uwierzytelnianie i przetwarzanie danych z zewnątrz.

Tabela 6. Wybrane scenariusze testowe i ich wynik oczekiwany

ID	Scenariusz	Wynik oczekiwany	Test
S1	Pełny cykl odnowienia: certyfikat → skaner → zadanie → zaproszenie → odnowienie → raport	wszystkie kroki zapisują zdarzenia; stary certyfikat w archiwum, nowy powiązany łańcuchem	EndToEndFlowTest
S2	Skaner uruchomiony dwa razy tego samego dnia	brak duplikatów zadań; priorytet rośnie wraz ze zbliżaniem się daty	RenewalScannerTest
S3	Operator, informatyk i pracownik przeglądają listy	każdy widzi wyłącznie rekordy ze swojego zakresu; osobne liczniki pulpitu	RoleScopeTest
S4	Zapis osoby bez firmy i certyfikatu kwalifikowanego bez osoby	odrzucone przez usługę (błąd przy polu) i przez bazę (wyjątek)	CompanyIntegrityTest
S5	Rejestracja, potwierdzenie adresu, logowanie, blokada po błędnych hasłach	konto nieaktywne do czasu potwierdzenia; blokada po 5 próbach; ta sama odpowiedź dla nieznanego adresu	AuthManagerTest
S6	Wiadomość z wnioskiem trafia do aplikacji dwukrotnie	powstaje jeden wniosek; firma uzupełniona z rejestru po NIP-ie	RegistrationIntakeTest
S7	Zatwierdzenie wniosku z błędem w danych certyfikatu	cała transakcja wycofana, błąd przy właściwym polu	RegistrationServiceTest
S8	Podgląd importu pliku CSV z błędnymi wierszami	podgląd pokazuje wynik zapisu, a w bazie nic nie przybywa	ImportServiceTest
S9	Eksport listy przez konto o ograniczonym zakresie	plik zawiera tylko rekordy w zakresie konta; w dzienniku zdarzenie eksportu	ExportServiceTest
S10	Migracje uruchomione na świeżym schemacie i na bazie ze starszego etapu	brak zaległych migracji; kształt bazy identyczny	MigrationRunnerTest

7.2. Testy funkcjonalne

Pełny zestaw liczy 394 testy: 209 jednostkowych i 185 integracyjnych, w sumie 52 klasy testowe. Tabele poniżej grupują je tematycznie wraz z liczbą przypadków i zakresem weryfikacji.

Tabela 7. Testy jednostkowe (209 przypadków)

Klasa testowa	Liczba	Co sprawdza
ApiKernelTest	6	kolejność kontroli w jądrze API: CSRF przed uwierzytelnieniem, 401, odczyt bez tokenu, zła metoda i JSON, mapowanie błędów usług na kody HTTP, brak szczegółów przy błędzie 500
AuthManagerTest	31	rejestracja, potwierdzenie adresu, jednorazowość i wygasanie tokenów, blokada po pięciu próbach, jednakowa odpowiedź dla błędnego hasła i nieznanego adresu, reset hasła, rehash, lista dozwolonych przekierowań, hasła tylko jako skróty
PasswordPolicyTest	8	polityka haseł wg NIST SP 800-63B (minimalna długość, lista haseł zakazanych)
OAuth2TokenProviderTest	4	wymiana refresh tokenu na token dostępu i jego pamięć podręczna
MailConfigTest, MailerRateLimitTest	7	konfiguracja sterowników poczty, ponowienie wysyłki po odrzuceniu przez limit dostawcy
RbacTest	12	macierz uprawnień siedmiu ról; nieznane uprawnienie lub rola oznacza brak dostępu
ValidatorTest, CertificateHelperTest	12	daty sprawdzane kalendarzem, NIP z sumą kontrolną, rabat procentowy, priorytety wg progów
TemplateRendererTest, TranslatorTest	10	podstawianie pól szablonu z escapowaniem, wersje językowe z domyślnym polskim
TranslationFilesTest	6	komplet kluczy w pięciu plikach językowych (polski, angielski, hiszpański, niemiecki, ukraiński) i brak pozostawionych tekstów angielskich
ExchangeFormatsTest, EmlParserTest	15	CSV (BOM, separator, ochrona przed formułami), XML, własny parser MIME wiadomości EML
ImapClientTest, WhiteListRegistryTest	18	klient IMAP na skryptowanym serwerze, odpowiedzi i błędy rejestru Białej Listy (w tym limit zapytań)
ViesRegistryTest, CompositeCompanyRegistryTest	17	numery VAT z Unii w systemie VIES (także państwa nieujawniające danych), kierowanie numeru do właściwego rejestru i odporność na awarię jednego źródła
KrsRegistryTest, CachedCompanyRegistryTest, LookupThrottleTest	18	uzupełnienie danych z Krajowego Rejestru Sądowego, pamięć podręczna odpowiedzi rejestru (bez zapamiętywania awarii) i limit zapytań na konto
RegistrationExtractorTest	19	odczyt wniosku z formularza, tabeli HTML i zwykłego zdania, polskie i angielskie etykiety, NIP z sumą kontrolną, odporność na wrogi HTML i przekroczone limity rozmiaru
CacheTest, ReportScheduleTest	8	pamięć podręczna agregatów i grupowanie harmonogramu wygaśnięć
TarArchiveTest, CodeDeployerTest	18	własny czytnik archiwum tar.gz (nagłówki GNU i pax, uszkodzone dane) oraz reguły wdrożenia: pliki chronione i pomijane, odrzucenie ścieżek wychodzących poza katalog aplikacji

Tabela 8. Testy integracyjne na jednorazowej bazie MySQL (185 przypadków)

Klasa testowa	Liczba	Co sprawdza
EndToEndFlowTest	2	cały cykl odnowienia przez warstwę API (ewidencja → skaner → zaproszenie → odnowienie → karta → eksport → import → dziennik) oraz odrzucenie złej metody, braku CSRF i niedozwolonej roli
RenewalScannerTest, TaskServiceTest	9	zadania tylko dla certyfikatów w marginesie, eskalacja priorytetu, brak duplikatów, statusy i odnowienie certyfikatu
InvitationServiceTest, TemplateAttachmentSettingsTest	8	wysyłka z szablonu i załącznikami, rejestr, przypomnienia wg ustawień, biblioteka plików
CertificateServiceTest, BeneficiaryServiceTest, PayerServiceTest	27	walidacja pól, unikalność, archiwizacja i przywracanie z zachowaniem powiązań, łańcuch odnowień oraz pobranie danych firmy z rejestru po NIP-ie wraz z limitem zapytań
CompanyIntegrityTest	7	łańcuch firma → osoba → certyfikat kwalifikowany wymuszany przez usługi i bazę; migracja naprawiająca stare dane
RoleScopeTest, AccountRolesTest, AdminFullAccessTest	19	zakres danych i uprawnienia każdej z siedmiu ról, osobne liczniki pulpitu dla ról
CompanyFilterTest	11	filtr firm: wszystkie / jedna / lista, wpływ na listy i pulpit, pominięcie przy eksporcie

Klasa testowa	Liczba	Co sprawdza
AccountServiceTest	5	zakładanie i wyłączanie kont, ochrona ostatniego administratora, przekazanie rekordów
RegistrationIntakeTest, RegistrationServiceTest, RegistrationApiTest	38	wnioski z e-maila: firma z rejestru, dopasowanie osób, duplikaty, kanały odbioru, zatwierdzenie w jednej transakcji z wycofaniem
NotificationServiceTest	12	wiadomości, wątki, prośby załatwione, komunikaty systemowe, zakres odbiorców
ReportServiceTest, SearchServiceTest, EventJournalTest	18	karty raportowe, wyszukiwarka z powodem dopasowania, oś czasu i dziennik administratora
ExportServiceTest, ImportServiceTest, EmlImportServiceTest	21	eksport w zakresie danych konta, import z podglądem wycofywanym transakcją i uzupełnianiem danych firmy z rejestru, dopasowanie wiadomości EML
MigrationRunnerTest, DashboardCacheTest	8	idempotentne migracje, zgodność schematu, pamięć podręczna z kluczem roli

Jedyny test przechodzący całą aplikację od początku do końca (EndToEndFlowTest) wykonuje kolejno: założenie firmy, osoby i certyfikatu, uruchomienie skanera, wysłanie zaproszenia, odnowienie certyfikatu, otwarcie karty raportowej, eksport do CSV, import pliku z podglądem oraz odczyt dziennika zdarzeń i wyszukiwarki — każdy krok jako żądanie do API z tokenem CSRF i sesją konkretnej roli.

7.3. Testy integracyjne i bezpieczeństwa logowania

Bezpieczeństwo sprawdzano trzema metodami: testami automatycznymi, przeglądem kodu pod kątem listy OWASP Top 10 [5] oraz kontrolą przez HTTP na uruchomionym serwerze Apache. Moduł logowania ma największy pojedynczy zestaw (31 przypadków w AuthManagerTest), który obejmuje m.in.: konto niepotwierdzone nie loguje się, a poprawne hasło na takim koncie nie wlicza się do limitu; link potwierdzający działa tylko raz i wygasa; wymyślony token jest odrzucany; rejestracja na zajęty adres niczego nie zmienia i wygląda tak samo; jednakowa odpowiedź dla błędnego hasła i nieznanego adresu; blokada po pięciu nieudanych próbach i jej zdjęcie po ustawieniu nowego hasła; wyłączone konto i konto bez hasła nie zalogują się; nowe hasło unieważnia pozostałe linki i uniemożliwia użycie starego hasła; przekierowanie po logowaniu akceptuje tylko znane cele; hasła są zapisywane wyłącznie jako skróty.

Tabela 9. Kontrole bezpieczeństwa według OWASP Top 10:2021 [5]

Ryzyko	Zastosowane zabezpieczenie	Weryfikacja
A01 Broken Access Control	Rbac + Visibility w każdej usłudze, zakres danych liczony z roli; ukrycie przycisków nie jest kontrolą	RbacTest, RoleScopeTest, EndToEndFlowTest (403), sprawdzenie HTTP
A02 Cryptographic Failures	password_hash(), tokeny 256-bitowe zapisywane jako SHA-256, cookie HttpOnly i SameSite=Lax (Secure przy HTTPS)	AuthManagerTest (hasła tylko jako skróty, rehash)
A03 Injection	PDO bez emulacji, escapowanie treści szablonów, flagi JSON_HEX_* w danych startowych, apostrof przed formułami w CSV	ValidatorTest, TemplateRendererTest, ExchangeFormatsTest
A04 Insecure Design	limity prób i wysyłek liczone w bazie, jednakowe komunikaty, archiwizacja zamiast usuwania, zatwierdzenie wniosku w transakcji	AuthManagerTest, RegistrationServiceTest
A05 Security Misconfiguration	.htaccess blokuje katalogi wewnętrzne, nagłówki Cache-Control, brak zależności od CDN	kontrola HTTP: 403 dla classes/, config/, storage/, .git/
A07 Identification and Authentication Failures	polityka haseł NIST, blokada po 5 błędach, potwierdzanie adresu, jednorazowe linki z terminem ważności	AuthManagerTest, PasswordPolicyTest

Ryzyko	Zastosowane zabezpieczenie	Weryfikacja
A08 Software and Data Integrity Failures	token CSRF (hash_equals) przy każdym zapisie, kontrola zawartości przesyłanych plików	ApiKernelTest, TemplateAttachmentSettingsTest
A09 Security Logging and Monitoring Failures	każdy zapis biznesowy w tabeli events, dziennik administratora, logi crona i poczty	EventJournalTest
A10 Server-Side Request Forgery	klient rejestru VAT buduje adres wyłącznie ze stałego hosta i dziesięciu cyfr NIP	WhiteListRegistryTest

Kontrola przez HTTP (przeprowadzona na serwerze Apache z rzeczywistą dostawą wiadomości do skrzynki Mailtrap) obejmowała cały przebieg: rejestracja, zablokowane logowanie przed potwierdzeniem, link pobrany ze skrzynki, logowanie, pulpit, wylogowanie, limit prób, brak tokenu CSRF, „zapomniałem hasła", nowe hasło i odrzucenie starego. Ścieżki wewnętrzne (classes, config, storage, tests, .git) zwracają 403, strony wejściowe i zasoby — 200, a API bez sesji — 401.

7.4. Ocena użyteczności interfejsu dashboardu

Ocenę użyteczności przeprowadzono dwiema metodami. Pierwszą jest ekspercka ocena heurystyczna według dziesięciu heurystyk Nielsena [7, 23], wykonana przez autora na działającej aplikacji; ma ona charakter subiektywny i nie zastępuje badania z użytkownikami. Wyniki zawiera tabela.

Tabela 10. Ocena heurystyczna interfejsu (ocena własna autora)

Heurystyka	Obserwacja w aplikacji	Ocena
Widoczność stanu systemu	szkielety treści w czasie ładowania, powiadomienia o wyniku operacji, licznik zadań i nieprzeczytanych wiadomości w menu	spełniona
Zgodność z rzeczywistością	polskie nazwy z dziedziny (firma, użytkownik certyfikatu, odnowienie), daty i kwoty w formacie polskim	spełniona
Kontrola i swoboda użytkownika	cofnięcie zadania do „do zrobienia", przywracanie z archiwum, podgląd importu bez zapisu	spełniona
Spójność i standardy	wspólne komponenty (tabele, okna, formularze, plakietki statusów), jednolite kolory priorytetów	spełniona
Zapobieganie błędom	walidacja pól po stronie serwera, blokada archiwizacji rekordów z powiązaniami, zatwierdzanie wniosków w transakcji	spełniona
Rozpoznawanie zamiast przypominania	wyszukiwarka globalna z powodem dopasowania, filtry list, podpowiadanie firmy po wyborze osoby	spełniona
Elastyczność i efektywność	skrót Ctrl+K, filtr firm, eksport z każdej listy; brak skrótów dla pozostałych akcji	częściowo
Estetyczny i minimalistyczny projekt	karty KPI i tabele bez zbędnych elementów; gęsty ekran szczegółów certyfikatu	częściowo
Pomoc w rozpoznawaniu i naprawie błędów	komunikaty przy polach formularza w języku użytkownika, wskazanie istniejącego NIP-u	spełniona
Pomoc i dokumentacja	instrukcja obsługi jest poza aplikacją (plik w repozytorium), brak pomocy kontekstowej w interfejsie	częściowo

Drugą metodą jest ankieta System Usability Scale (SUS) [8], zawierająca dziesięć stwierdzeń oceniane na skali od 1 („zdecydowanie się nie zgadzam") do 5 („zdecydowanie się zgadzam"). Wynik jednej osoby oblicza się tak: dla stwierdzeń nieparzystych odejmuje się 1 od odpowiedzi, dla parzystych odejmuje się odpowiedź od 5, sumę mnoży się przez 2,5, a wynik (0–100) uśrednia po respondentach; wartość powyżej około 68 punktów uznaje się za wynik powyżej przeciętnej.

Tabela 11. Stwierdzenia ankiety SUS (wersja polska)

Nr	Stwierdzenie
1	Myślę, że chciałbym często korzystać z tego systemu.
2	Uważam, że system jest niepotrzebnie skomplikowany.
3	Uważam, że system jest łatwy w użyciu.
4	Myślę, że potrzebowałbym pomocy osoby technicznej, aby móc korzystać z tego systemu.
5	Uważam, że różne funkcje systemu są dobrze ze sobą zintegrowane.
6	Uważam, że w systemie jest zbyt wiele niespójności.
7	Wyobrażam sobie, że większość osób szybko nauczy się korzystać z systemu.
8	Uważam, że system jest bardzo niewygodny w użyciu.
9	Czułem się bardzo pewnie, korzystając z systemu.
10	Musiałem nauczyć się wielu rzeczy, zanim mogłem zacząć korzystać z systemu.

Tabela 12. Wyniki ankiety SUS — do uzupełnienia po przeprowadzeniu badania

Respondent	Rola w badaniu	Wynik SUS (0–100)
R1	[do uzupełnienia]	[do uzupełnienia]
R2	[do uzupełnienia]	[do uzupełnienia]
R3	[do uzupełnienia]	[do uzupełnienia]
Średnia	—	[do uzupełnienia]

7.5. Analiza uzyskanych wyników

Tabela 13. Podsumowanie wskaźników jakości

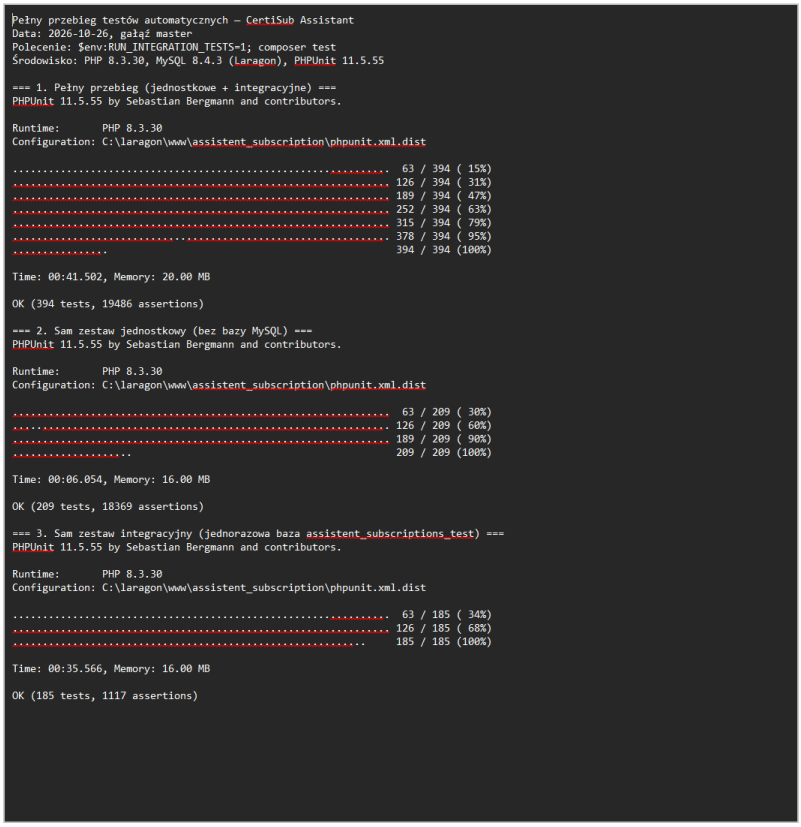
Wskaźnik	Wartość	Źródło / data
Liczba testów	394 (209 jednostkowych + 185 integracyjnych)	PHPUnit 11.5, 26.10.2026
Wynik pełnego przebiegu z testami integracyjnymi	394 z 394 zaliczonych, 19 486 asercji, czas 42 s	przebieg z 26.10.2026
Przebieg bez testów integracyjnych	209 wykonanych, 18 369 asercji, 185 pominiętych	PHPUnit, 26.10.2026
Analiza statyczna PHPStan (poziom 5)	brak błędów	composer stan, 26.10.2026
Styl kodu PHP-CS-Fixer	0 uwag w 210 plikach	composer cs, 26.10.2026
Zgodność schematu	190 kolumn, 87 pozycji indeksów, 32 klucze obce, 2 ograniczenia CHECK — identyczne w bazie aplikacji i bazie testowej	porównanie information_schema, 2.10.2026
Pokrycie wymagań funkcjonalnych	19 z 19 (F1–F19)	macierz w docs/MAPA_PROJEKTU.md
Czas odpowiedzi API kart i wyszukiwarki	poniżej 50 ms na danych demonstracyjnych	pomiar lokalny, 17.09.2026

Wyniki potwierdzają, że wszystkie wymagania funkcjonalne z opisu pracy zostały zrealizowane, a krytyczne reguły (uprawnienia, spójność danych, uwierzytelnianie) są chronione testami automatycznymi. Największą wartość przyniosły testy integracyjne na prawdziwej bazie — ujawniły m.in. ograniczenie MySQL, które nie pozwala łączyć klauzuli CHECK z akcją CASCADE na tej samej kolumnie klucza obcego (błąd 3823), co wymusiło zmianę reguły aktualizacji klucza beneficiary_id na RESTRICT. Z kolei rozjazd stref czasowych PHP i MySQL, który przesuwał priorytety o jeden dzień, wykryto przy przygotowywaniu danych demonstracyjnych i usunięto ustawieniem strefy Europe/Warsaw w bootstrap.php.

Ograniczenia obecnego stanu testów: scenariusz end-to-end przechodzi przez API, a nie przez przeglądarkę (brak testów Selenium/Playwright); nie przeprowadzono testów

obciążeniowych ani badania użyteczności z użytkownikami (tabela wyników SUS czeka na uzupełnienie); testy nie obejmują wysyłki przez prawdziwy serwer SMTP, bo żaden test nie może wysyłać prawdziwych wiadomości — tę ścieżkę sprawdzono ręcznie w środowisku sandbox Mailtrap.

Osobnej uwagi wymaga klasa błędów, których zestaw testów nie mógł wykryć, bo leżą poza kodem aplikacji — w jej konfiguracji na konkretnym serwerze. Przy pierwszym wdrożeniu wersji demonstracyjnej narzędziem pobierającym kod z repozytorium (26.10.2026) nadpisany został plik config/database.php: w repozytorium zawiera on ustawienia lokalne, a na hostingu przechowywał dane logowania do bazy. Strona startowa, która nie korzysta z bazy, działała dalej, natomiast logowanie i całe API zaczęły zwracać kod 500. Wszystkie 394 testy przechodziły, bo każdy z nich otrzymuje połączenie z bazą testowej konfiguracji — żaden nie opisuje tego, co wolno nadpisać przy wdrożeniu. Poprawka objęła trzy elementy: plik trafił na listę chronionych, dane różniące się między serwerami wyniesiono do pliku config/database.local.php poza repozytorium, a regułę zabezpieczono testem CodeDeployerTest, który sprawdza, że wdrożenie zostawia produkcyjną konfigurację nietkniętą. Wniosek ogólny: granica między kodem a konfiguracją środowiska jest częścią interfejsu systemu i powinna być opisana testem tak samo jak reguły biznesowe.



Rysunek 34. Zapis pełnego przebiegu testów automatycznych (docs/testy/przebieg-testow-2026-10-26.txt)

8. Dokumentacja użytkownika

Rozdział opisuje instalację i obsługę aplikacji. Pełna instrukcja wraz ze scenariuszem demonstracji (ok. 30 minut) znajduje się w repozytorium w pliku docs/INSTRUKCJA_OBSLUGI.md, a uruchomienie na bezpłatnym hostingu — w docs/DEPLOY_INFINITYFREE.md.

8.1. Wymagania i uruchomienie

Wymagania wstępne środowiska lokalnego:

- PHP 8.1 lub nowszy z rozszerzeniami PDO i MySQL, MySQL 8 lub MariaDB 10.5+, Composer,

- Apache z modułem mod_rewrite i AllowOverride All (np. Laragon); katalog aplikacji jest jednocześnie katalogiem głównym serwera,

- Node.js — opcjonalnie, tylko do przebudowy arkusza stylów (zbudowany plik jest w repozytorium).

Kroki instalacji:

- composer install — instaluje zależności,

- config/database.php — dane połączenia z MySQL (domyślnie localhost, baza assistent_subscriptions, użytkownik root),

- config/mail.local.php (wzorzec: config/mail.local.php.example) — poczta; sterownik sandbox kieruje wiadomości wyłącznie do skrzynki Mailtrap, a sterownik log zapisuje je do logs/mail.log,

- php scripts/migrate.php — tworzy i aktualizuje schemat (nową bazę można utworzyć opcją --fresh, która usuwa wszystkie tabele — wyłącznie dla bazy tymczasowej),

- php scripts/seed-demo-data.php — opcjonalne dane demonstracyjne (konta demo mają ustawione hasła),

- adres aplikacji: http://localhost/assistent_subscription/.

Zadania okresowe: raz na dobę php cron/renewals.php (skaner i przypomnienia), a co kilka minut — jeśli włączono odbiór wniosków ze skrzynki — php cron/mail-intake.php.

8.2. Logowanie, konta i role

Logowanie odbywa się adresem e-mail i hasłem na stronie login.php. Konto założone przez administratora nie ma hasła — osoba dostaje wiadomość z linkiem (ważnym 2 godziny) i sama je ustawia. Przy włączonej rejestracji publicznej nowe konto trzeba potwierdzić linkiem z wiadomości (ważnym 24 godziny). Zapomniane hasło odzyskuje się linkiem „Nie pamiętasz hasła?”. Menu i dostępne akcje zależą od roli (tabela w podrozdziale 3.4 i załącznik B).

8.3. Praca z ewidencją

- Certyfikaty: lista z wyszukiwarką i filtrami, szczegóły z historią i łańcuchem odnowień, formularz dodawania. Certyfikat kwalifikowany wymaga użytkownika certyfikatu, a ten — firmy. Rabat wpisuje się jako procent (np. -5%). Archiwizacja (menedżer, administrator)

ukrywa rekord z bieżących list i zamyka otwarte zadanie jako porzucone; z archiwum rekord można przywrócić.

- Użytkownicy certyfikatów i firmy: osoby nie można zarchiwizować, dopóki ma bieżące certyfikaty, a firmy — dopóki ma osoby lub certyfikaty. NIP jest sprawdzany sumą kontrolną i musi być unikalny.

- Wyszukiwarka globalna (Ctrl+K lub /): wpisanie co najmniej dwóch znaków podpowiada certyfikaty, osoby i firmy z podaniem powodu dopasowania; można szukać po nazwie, numerze seryjnym, NIP-ie (kreski i spacje nie mają znaczenia), telefonie lub nazwisku opiekuna.

- Filtr firm (pasek „Zakres firm”): zawęża pracę do wszystkich firm, jednej firmy lub wybranej listy; nie jest kontrolą dostępu, a eksport go pomija.

8.4. Proces odnowienia krok po kroku

- Skaner (codziennie przez cron albo przyciskiem „Uruchom skaner”) zakłada zadanie dla certyfikatu w marginesie odnowienia — pojawia się na liście ToDo z priorytetem.

- Osoba prowadząca sprawę klika „Rozpocznij” (status „w toku”) i „Wyślij zaproszenie”: wybiera odbiorcę, szablon i załączniki, sprawdza podgląd i wysyła.

- W rejestrze „Zaproszenia” widać status wysyłki; przyciski „Przypomnij”, „Ponów”, „Odpowiedziano” i „Zamknij” obsługują dalszy kontakt, a cron wysyła kolejne przypomnienia (domyślnie co 7 dni, do 3).

- Po załatwieniu sprawy w panelu zadania klika się „Odnów certyfikat”, podając nową datę wygaśnięcia (późniejszą od poprzedniej), numer seryjny i płatność; stary certyfikat trafia do archiwum, a zadanie otrzymuje status „zrobione”. Zadanie można też porzucić, podając powód.

8.5. Raporty, wymiana danych i wnioski e-mail

- Raporty: karta użytkownika certyfikatu i karta firmy (daty odnowienia, certyfikaty z archiwum, zadania, zaproszenia, oś czasu), harmonogram wygaśnięć na 3–24 miesiące; przycisk „Drukuj” tworzy wydruk bez menu.

- Eksport (menedżer i wyżej, szef, księgowa): przyciski CSV i XML przy listach, kartach i w widoku „Import i eksport”; CSV otwiera się w Excelu bez kreatora.

- Import (administrator): kolejność firmy → użytkownicy certyfikatów → certyfikaty; krok „Sprawdź plik” pokazuje wynik dla każdego wiersza (nowy, aktualizacja, bez zmian, pominięty, błąd), a „Importuj” zapisuje poprawne wiersze; do pobrania są wzory plików.

- Wnioski e-mail (operator, menedżer, administrator): wczytanie pliku .eml lub odbiór ze skrzynki tworzy gotowy formularz; operator sprawdza dane, w razie potrzeby prosi o uzupełnienie lub odrzuca, a „Zatwierdź i utwórz” zakłada firmę, osobę i certyfikat.

- Powiadomienia: wiadomości między kontami, prośby o uzupełnienie danych powiązane z rekordem i komunikaty systemowe; liczba nieprzeczytanych widoczna jest w menu.

8.6. Administracja

- Konta i role: dodawanie kont z rolą, edycja, przekazanie rekordów innemu kontu (np.

przed odejściem pracownika), wysłanie linku do hasła, wyłączanie i włączanie kont; nie można wyłączyć własnego konta ani ostatniego administratora.

- Szablony i załączniki: edycja treści z podglądem, język, aktywność; biblioteka plików do 10 MB.

- Ustawienia: margines odnowienia (30 dni), próg krytyczny (7 dni), odstęp między przypomnieniami (7 dni), limit przypomnień (3).

- Dziennik zdarzeń: wszystkie operacje z filtrami (szukaj, obszar, zdarzenie, autor, daty); tylko do odczytu.

8.7. Rozwiązywanie problemów

Tabela 14. Najczęstsze problemy i ich rozwiązanie

Objaw	Przyczyna i rozwiązanie
Strona się nie otwiera lub brak połączenia z bazą	Apache lub MySQL nie działa — uruchomić usługi (Laragon: Start All), potem php scripts/migrate.php
Błąd o brakującej tabeli lub kolumnie	kod i baza pochodzą z różnych etapów — php scripts/migrate.php
Panel pusty, wszędzie zera	brak danych — php scripts/seed-demo-data.php --force
Nie widzę menu Archiwum lub Konta i role	rola konta nie ma tych uprawnień (archiwum od menedżera, konta tylko administrator) — administrator zmienia rolę
Zaproszenie ma status „Nieudane”	błąd serwera pocztowego (treść w rejestrze) — sprawdzić config/mail.local.php, potem „Ponów”
Nie przychodzi wiadomość z linkiem	przy sterowniku sandbox sprawdzić skrzynkę Mailtrap; sprawdzić, czy konto nie jest wyłączone i czy nie przekroczono limitu wysyłek
Skaner nie zakłada zadania	certyfikat jest poza marginesem, w archiwum, ma otwarte zadanie albo zadanie dla tej daty zostało zamknięte
Pulpit pokazuje stare liczby	wskaźniki liczone są raz na 5 minut, a zapis danych je odświeża; sprawdzić prawa zapisu do storage/cache
Import: „Nie znaleziono płatnika”	najpierw zaimportować firmy, a w pliku osób i certyfikatów podać NIP lub dokładną nazwę
Testy integracyjne są pomijane	ustawić RUN_INTEGRATION_TESTS=1 (wymaga działającego MySQL)

9. Zakończenie

9.1. Podsumowanie pracy

Celem pracy było zaprojektowanie i zaimplementowanie aplikacji webowej wspierającej ewidencję certyfikatów (w tym kwalifikowanych) wraz z ich użytkownikami i płatnikami, nadzorującej terminy odnowień i obsługującej korespondencję z beneficjentami. Cel został osiągnięty: powstał działający system CertiSub Assistant, który realizuje wszystkie dziewiętnaście wymagań funkcjonalnych z opisu pracy oraz wymagania niefunkcjonalne dotyczące technologii (PHP, HTML, JavaScript, MySQL w standardowym układzie LAMP), bezpieczeństwa i możliwości wdrożenia na zwykłym hostingu.

Najważniejsze rezultaty pracy:

- ewidencja certyfikatów ośmiu rodzajów z użytkownikami i firmami, archiwizacją zamiast usuwania oraz spójnością danych wymuszaną przez bazę (łańcuch firma → użytkownik → certyfikat kwalifikowany),
- automatyczny proces odnowień: skaner, lista ToDo z priorytetami i statusami, zaproszenia z szablonów z załącznikami, przypomnienia, odnowienie z łańcuchem certyfikatów i statystyki zadań,
- raporty perspektyw użytkownika, płatnika i administratora, wyszukiwarka z powiązaniami, oś czasu i dziennik zdarzeń, import i eksport CSV, XML i EML,
- siedem ról stanowisk z zakresem danych liczonym po stronie serwera oraz logowanie hasłem z potwierdzeniem adresu, limitami prób i jednorazowymi tokenami,
- moduł wniosków e-mail z odczytem danych, uzupełnianiem firmy z rejestru VAT i zatwierdzaniem w jednej transakcji oraz powiadomienia wewnętrzne,
- 394 testy automatyczne, analiza statyczna na poziomie 5 bez błędów i dokumentacja (instrukcja obsługi, opis bazy danych, opis API i poleceń CLI),
- wersja demonstracyjna wdrożona na bezpłatnym hostingu bez dostępu SSH.

Wybór stosu PHP/LAMP, uzasadniony studium wykonalności, okazał się trafny: aplikacja działa na zwykłym hostingu bez kroku budowania, a brak zależności od CDN i rozszerzeń PHP pozwolił uruchomić ją także w sieci wewnętrznej. Decyzje projektowe podejmowane w trakcie prac — wydzielenie modułu prywatnego do osobnej aplikacji, zamiana ceny na rabat procentowy, rozszerzenie ról do profili stanowisk, przejście z kodów OTP na hasła — ilustrują, że wymagania doprecyzowywały się wraz z poznawaniem dziedziny, a architektura warstwowa pozwoliła je wprowadzać bez przebudowy.

Ograniczenia: dostęp do systemu mają wyłącznie konta personelu (użytkownicy certyfikatów i płatnicy nie logują się, a ich perspektywy są raportami dla personelu — decyzja D2); testy nie obejmują interfejsu w przeglądarce ani obciążenia; powiadomienia dla użytkowników końcowych ograniczają się do wiadomości e-mail; odbiór wniosków wymaga skonfigurowania skrzynki IMAP albo webhooka.

Wdrożenie wersji demonstracyjnej dostarczyło też wniosku, którego nie dało się wyprowadzić z samego kodu. Hosting bez dostępu SSH wymusił narzędzie pobierające

paczkę z repozytorium wprost na serwer, a pierwsze uruchomienie tego narzędzia nadpisało konfigurację połączenia z bazą danych i unieruchomiło logowanie (opis w podrozdziale 7.5). Przyczyną nie była pomyłka w pojedynczej linii, lecz brak jawnie zapisanej granicy między kodem a konfiguracją środowiska: plik należał jednocześnie do repozytorium i do ustawień serwera. Po poprawce dane zależne od środowiska mieszkają w plikach spoza repozytorium, a reguła jest pilnowana testem. Doświadczenie to potwierdza, że przy aplikacji przeznaczonej na zwykły hosting proces wdrożenia wymaga takiej samej uwagi projektowej jak sama aplikacja.

9.2. Kierunki dalszego rozwoju

- osobne logowanie dla użytkowników certyfikatów i płatników z dostępem tylko do własnej karty (samoobsługowy wgląd w daty odnowienia i odpowiedź na zaproszenie jednym kliknięciem),
- testy interfejsu w przeglądarce (Playwright) i testy obciążeniowe,
- uwierzytelnianie dwuskładnikowe dla kont o rolach z szerokim zakresem danych oraz integracja z zewnętrznym dostawcą tożsamości (SSO),
- powiadomienia w dodatkowych kanałach (SMS, komunikatory) i kalendarz wygaśnięć w formacie iCalendar,
- automatyczne pobieranie informacji o statusie certyfikatów SSL z publicznych rejestrów przejrzystości certyfikatów oraz integracja z interfejsami wystawców,
- kolejne źródła danych o firmach, które wymagają wniosku o bezpłatny klucz dostępu: CEIDG (działalności jednoosobowe) i GUS BIR/REGON — warstwa CompanyRegistry jest na nie przygotowana, brakuje wyłącznie poświadczeń,
- rozszerzenie odczytu wniosków o załączniki PDF (OCR) i uczenie ekstraktora na własnych wiadomościach,
- powiązanie z osobną aplikacją „Menedżer Subskrypcji” (prywatne subskrypcje) na poziomie wspólnego logowania, przy zachowaniu rozdzielnych baz danych,
- zmiana katalogu głównego serwera na osobny katalog public/ — wtedy blokady w pliku .htaccess przestaną być potrzebne.

Bibliografia

[1] Rozporządzenie Parlamentu Europejskiego i Rady (UE) nr 910/2014 z dnia 23 lipca 2014 r. w sprawie identyfikacji elektronicznej i usług zaufania w odniesieniu do transakcji elektronicznych na rynku wewnętrznym oraz uchylające dyrektywę 1999/93/WE (eIDAS), Dz. Urz. UE L 257 z 28.8.2014.

[2] Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 z dnia 27 kwietnia 2016 r. w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych (RODO), Dz. Urz. UE L 119 z 4.5.2016.

[3] Ustawa z dnia 5 września 2016 r. o usługach zaufania oraz identyfikacji elektronicznej (Dz. U. z 2016 r. poz. 1579).

[4] Grassi P. A., Garcia M. E., Fenton J. L., Digital Identity Guidelines: Authentication and Lifecycle Management, NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017.

[5] OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks, OWASP Foundation, 2021.

[6] OWASP Application Security Verification Standard 4.0, OWASP Foundation, 2019.

[7] Nielsen J., Usability Engineering, Academic Press, Boston 1993.

[8] Brooke J., SUS: A “quick and dirty” usability scale, [w:] Jordan P. W. i in. (red.), Usability Evaluation in Industry, Taylor & Francis, London 1996, s. 189–194.

[9] Fowler M., Patterns of Enterprise Application Architecture, Addison-Wesley, Boston 2002.

[10] Sommerville I., Software Engineering, wyd. 10, Pearson, Harlow 2015.

[11] PHP Manual, The PHP Group, https://www.php.net/manual (dostęp: październik 2026).

[12] MySQL 8.4 Reference Manual, Oracle Corporation, https://dev.mysql.com/doc/refman/8.4/en/ (dostęp: październik 2026).

[13] Vue.js — The Progressive JavaScript Framework, dokumentacja, https://vuejs.org (dostęp: październik 2026).

[14] Tailwind CSS — dokumentacja, https://tailwindcss.com/docs (dostęp: październik 2026).

[15] PHPMailer — A full-featured email creation and transfer class for PHP, https://github.com/PHPMailer/PHPMailer (dostęp: październik 2026).

[16] PHPUnit Manual, https://docs.phpunit.de (dostęp: październik 2026).

[17] PHPStan — PHP Static Analysis Tool, dokumentacja, https://phpstan.org (dostęp: październik 2026).

[18] Biała lista podatników VAT — interfejs API, Ministerstwo Finansów, https://www.gov.pl/web/kas/api-wykazu-podatnikow-vat (dostęp: październik 2026).

[19] Crispin M., RFC 3501: Internet Message Access Protocol — Version 4rev1, IETF, 2003.

[20] Resnick P., RFC 5322: Internet Message Format, IETF, 2008.

[21] Hardt D., RFC 6749: The OAuth 2.0 Authorization Framework, IETF, 2012.

[22] Spring Boot Reference Documentation, VMware / Broadcom, https://docs.spring.io/spring-boot (dostęp: październik 2026).

[23] Nielsen J., 10 Usability Heuristics for User Interface Design, Nielsen Norman Group, https://www.nngroup.com/articles/ten-usability-heuristics/ (dostęp: październik 2026).

[24] VIES — VAT Information Exchange System, interfejs REST, Komisja Europejska, https://ec.europa.eu/taxation_customs/vies/ (dostęp: październik 2026).

[25] Krajowy Rejestr Sądowy — interfejs API, Ministerstwo Sprawiedliwości, https://api-krs.ms.gov.pl (dostęp: październik 2026).

Spis rysunków

Rysunek 1. Architektura warstwowa aplikacji CertiSub Assistant ... 18
Rysunek 2. Diagram ERD bazy danych assistent_subscriptions (stan po Etapie 10) ... 20
Rysunek 3. Lista tabel bazy w phpMyAdmin (wersja demonstracyjna) ... 23
Rysunek 4. Zawartość tabeli certificates w phpMyAdmin (dane demonstracyjne) ... 23
Rysunek 5. Strona główna aplikacji — nagłówek z opisem funkcji ... 27
Rysunek 6. Strona główna — rodzaje śledzonych pozycji i opis działania ... 28
Rysunek 7. Strona główna — przegląd funkcji i zaproszenie do logowania ... 28
Rysunek 8. Pulpit administratora: wskaźniki KPI, liczniki statusów i statystyki zadań ... 29
Rysunek 9. Lista certyfikatów z filtrami, rabatem, płatnością i statusem ... 29
Rysunek 10. Lista użytkowników certyfikatów — każda osoba należy do firmy ... 30
Rysunek 11. Formularz dodawania certyfikatu (sekcje: dane, ważność, powiązania, rabat i płatność) ... 31
Rysunek 12. Panel szczegółów certyfikatu: powiązania, sekcja „Odnowienie” i historia zdarzeń ... 32
Rysunek 13. Widok „Raporty”: perspektywy i harmonogram wygaśnięć ... 33
Rysunek 14. Karta firmy: dane kontaktowe, wskaźniki i powiązane osoby ... 33
Rysunek 15. Karta firmy: harmonogram wygaśnięć na 12 miesięcy ... 34
Rysunek 16. Karta firmy: certyfikaty z datami odnowienia, priorytetem i rabatem ... 34
Rysunek 17. Karta firmy: ścieżka realizacji, zaproszenia i historia na osi czasu ... 34
Rysunek 18. Pasek „Zakres firm” — okno wyboru: wszystkie firmy, jedna firma albo lista ... 35
Rysunek 19. Ekran logowania ... 35
Rysunek 20. Wiadomość „Ustaw hasło” w skrzynce Mailtrap (sandbox) ... 36
Rysunek 21. Ta sama wiadomość dostarczona do rzeczywistej skrzynki odbiorcy (wersja demonstracyjna) ... 36
Rysunek 22. Widok „Konta i role” dla administratora ... 37
Rysunek 23. Lista ToDo: zadanie o priorytecie „Krytyczne” po terminie ... 39
Rysunek 24. Widok „Ustawienia” — progi procesu odnowień ... 39
Rysunek 25. Ekran „Szablony i załączniki” — cztery szablony domyślne (PL i EN) ... 40
Rysunek 26. Okno „Wyślij zaproszenie”: wybór odbiorcy, szablon i podgląd treści z podstawionymi danymi certyfikatu ... 41
Rysunek 27. Rejestr „Zaproszenia” z filtrami statusu i zaległych przypomnień ... 41
Rysunek 28. Zaproszenie dostarczone do skrzynki odbiorcy z wersji demonstracyjnej na hostingu ... 42
Rysunek 29. Widok „Import i eksport” — eksport zbiorów i kreator importu ... 43
Rysunek 30. Dziennik zdarzeń administratora z filtrami ... 43
Rysunek 31. Kolejka „Wnioski e-mail” (zakładki: do sprawdzenia, zatwierdzone, odrzucone) ... 44
Rysunek 32. Ekran sprawdzenia wniosku: treść wiadomości i lista uwag obok formularza — w tej wiadomości nie było NIP-u, więc danych firmy nie dało się pobrać z rejestru ... 44
Rysunek 33. Widok „Powiadomienia” — prośby o uzupełnienie danych ... 46
Rysunek 34. Zapis pełnego przebiegu testów automatycznych (docs/testy/przebieg-testow-2026-10-26.txt) ... 52

Spis tabel

Tabela 1. Role stanowisk, zakres danych i najważniejsze uprawnienia ... 16
Tabela 2. Tabele bazy danych i ich przeznaczenie ... 21
Tabela 3. Technologie i narzędzia wykorzystane w projekcie ... 25
Tabela 4. Statusy zadania odnowienia i dozwolone przejścia (klasa TaskService) ... 38
Tabela 5. Statusy zaproszenia (tabela invitations) ... 40
Tabela 6. Wybrane scenariusze testowe i ich wynik oczekiwany ... 47
Tabela 7. Testy jednostkowe (209 przypadków) ... 47
Tabela 8. Testy integracyjne na jednorazowej bazie MySQL (185 przypadków) ... 48
Tabela 9. Kontrole bezpieczeństwa według OWASP Top 10:2021 [5] ... 49
Tabela 10. Ocena heurystyczna interfejsu (ocena własna autora) ... 50
Tabela 11. Stwierdzenia ankiety SUS (wersja polska) ... 50
Tabela 12. Wyniki ankiety SUS — do uzupełnienia po przeprowadzeniu badania ... 51
Tabela 13. Podsumowanie wskaźników jakości ... 51
Tabela 14. Najczęstsze problemy i ich rozwiązanie ... 55

Załączniki

Załącznik A. Oficjalny opis pracy (dosłownie)

Opis:
W bezpośredniej perspektywie jest to aplikacja webowa (lub stanowiskowa) do której możemy dodać informację o użytkownikach (dane osobowe) certyfikatu (powiązanego z usługą), certyfikacie (dane usługi), płatnikach (dane płatnika). System wiąże pozyskane informacje razem umożliwiając wykonanie raportowania w zależności od wymaganej perspektywy: Użytkownik – daty odnowienia, szczegóły certyfikatu oraz historia; Płatnik – lista certyfikatów, daty ich wygasania, historia, osoby powiązanel; Administrator – zarządzanie systemem niezależnie od perspektyw; System powinien składować dane w bazie, umożliwiać dodawanie nowych rekordów, edycję i archiwizację danych; eksport danych do pliku csv/xml oraz import takich danych z plików csv/xml/eml.

Składowe systemu:
System składać się będzie z:
– aplikacji webowej umożliwiającej wprowadzenie przez zarządzającego danych:
° certyfikatu (np. danych certyfikatów kwalifikowanych w postaci numeru seryjnego, daty ważności, wygasania, wymagań co do czasu odnowienia etc.),
° użytkowników certyfikatu (dane osobowe beneficjentów),
° płatników (beneficjent nie jest płatnikiem za usługę, ale jest z płatnikiem powiązany).

System ma za zadanie nadzorować składowane w Bazie Danych dane certyfikatów i poprzez ich regularne skanowanie wyszukiwać te, które winny być w założonym marginesie czasowym odnowione.

System winien:

– tworzyć listę “ToDo” (do zrobienia) z priorytetyzowaniem w zależności od założonych zakresów dat wygaśnięcia certyfikatów;
– umożliwiać wysłanie zaproszeń do odnowienia poprzez e-mail z wykorzystaniem szablonu wiadomości i załączników;
– zarządzanie wysłanymi zaproszeniami (przypomnienia).

i ponadto:
– implementować system dostępowy oparty o konta użytkowników,
– umożliwiać wyszukiwanie usług/użytkowników/płatników i istniejących powiązań pomiędzy nimi,
– umożliwiać tworzenie i przegląd ścieżki realizacji i powiązań na osi czasu pomiędzy elementami systemu,
– prowadzić statystyki realizacji zadań przy grupowaniu do ustalonych statusów, np: do zrobienia, zrobiony, porzucony etc.

Założenia dodatkowe:
System opierać się będzie na zarządzaniu dostępem do niego poprzez zhierarchizowany plan kont z podziałem na role.

Element badawczy pracy:
Praca zawierać będzie wstęp, ogólną charakterystykę problemu, analizę istniejących rozwiązań,

projekt ogólny i techniczny, dokumentację użytkownika, opis testów, zakończenie i podsumowanie pracy wraz z przedstawieniem ewentualnych przyszłych zmian rozwojowych.

Część implementacyjna:
Przewidywaną formą realizacji jest aplikacja webowa korzystająca z Bazy stworzonej jako system bazodanowy ulokowany w publicznie (lub intranetowo w przypadku instalacji w chronionym i ograniczonym dostępie) dostępnych zasobach internetowych (serwer Bazy Danych w ramach standardowej usługi hostingu). Bazodanowa warstwa serwerowa oparta będzie o środowisko zgodne z MySQL (LAMP).

Przewiduje się że narzędziem realizacji będzie Java i Spring. Jednak na etapie wstępnym nastąpi analiza dostępnych opcji z dyskusją rozwiązań i badaniem tematu, po studium wykonalności zostanie wyłonione narzędzie programistyczne z wykorzystaniem którego aplikacja zostanie zrealizowana. Webowa aplikacja bazodanowa stanowiąca jądro systemu przewidziana jest do realizacji z wykorzystaniem PHP, HTML i JavaScript.

Załącznik B. Macierz uprawnień ról

Macierz wygenerowano z pliku classes/Rbac.php. Znak ● oznacza, że rola ma dane uprawnienie. Skróty: ADM — administrator, DIR — szef, MGR — menedżer, ACC — księgowa, IT — informatyk, OPR — operator, EMP — pracownik.

Tabela 15. Macierz uprawnień ról stanowisk

Uprawnienie	ADM	DIR	MGR	ACC	IT	OPR	EMP
certificates.view	●	●	●	●	●	●	●
certificates.create	●		●		●	●	
certificates.update	●		●		●	●	
certificates.update_payment	●		●	●		●	
certificates.assign_owner	●	●	●				
certificates.archive	●		●				
beneficiaries.view	●	●	●	●	●	●	●
beneficiaries.create	●		●			●	
beneficiaries.update	●		●			●	
beneficiaries.archive	●		●				
payers.view	●	●	●	●	●	●	●
payers.create	●		●			●	
payers.update	●		●	●		●	
payers.archive	●		●				
archive.view	●	●	●				
tasks.view	●	●	●		●	●	●
tasks.update	●		●		●	●	
tasks.assign	●	●	●				
tasks.stats	●	●	●				
invitations.view	●	●	●		●	●	
invitations.send	●		●		●	●	
templates.manage	●						
attachments.manage	●						
scanner.run	●		●				
reports.view	●	●	●	●	●	●	●
search.use	●	●	●	●	●	●	●
events.view_all	●						
export.run	●	●	●	●			
import.run	●						
registrations.view	●	●	●			●	
registrations.review	●		●			●	
registrations.intake	●		●			●	
notifications.use	●	●	●	●	●	●	●
notifications.broadcast	●						
accounts.manage	●						
settings.manage	●						

Załącznik C. Struktura katalogów repozytorium

Tabela 16. Najważniejsze katalogi projektu

Katalog	Zawartość
api/	20 punktów wejścia API (każdy to jedna linia uruchamiająca kontroler)
classes/	kod aplikacji: Service, Api, Http, Exchange, Intake, Auth, Mail, Migrations
assets/	interfejs: js (core, components, views), css (Tailwind), vendor (Vue), fonts
cron/, scripts/	zadania okresowe i narzędzia CLI (migracje, dane demo, hasła, test poczty)
database/	schema.sql — schemat świeżej instalacji

Katalog	Zawartość
config/	konfiguracja bazy, autoryzacji, poczty i odbioru wniosków (sekrety w plikach *.local.php poza gitem)
lang/	tłumaczenia (pl, en, es, de, uk)
storage/	załączniki, oryginały wniosków, pamięć podręczna — zablokowane przez HTTP
tests/	testy jednostkowe, integracyjne i klasy pomocnicze
docs/	dokumentacja: mapa projektu, instrukcja, opis bazy, API, CLI, wdrożenie, diagramy ERD